

Το σημαντικότερο mental model της φάσης είναι:

</> Plain text

```
WHO?
user / group / service principal
  |
  ▼
WHAT OBJECT?
metastore
├── catalog
│   └── schema
│       ├── table
│       ├── view
│       ├── volume
│       ├── function
│       └── model
  |
  ▼
WHAT ACTION?
USE / SELECT / MODIFY / CREATE / READ VOLUME / ...
  |
  ▼
WHERE ARE THE BYTES?
managed storage
or
external location → cloud object storage
```

Αν αυτό γίνει καθαρό, σχεδόν όλο το Unity Catalog αρχίζει να βγάζει νόημα.

1. Γιατί υπάρχει το Unity Catalog;

Ας ξεκινήσουμε χωρίς Databricks.

Έστω εταιρεία με:

</> Plain text

```
S3 / ADLS
|
├── finance/
│   ├── payroll/
│   └── invoices/
|
├── sales/
│   ├── orders/
│   └── customers/
|
└── marketing/
```

Και έχεις 100 developers.

Χωρίς central governance, πρέπει να λύσεις ερωτήσεις όπως:

</> Plain text

```
Ποιος μπορεί να διαβάσει payroll;

Ποιος μπορεί να γράψει orders;

Ποιος είναι owner;

Από ποιο pipeline δημιουργήθηκε αυτό το table;

Ποιο dashboard χρησιμοποιεί αυτή τη στήλη;

Ποιος service principal έχει access;

Τι γίνεται αν κάποιος διαβάσει απευθείας
το storage path παρακάμπτοντας permissions;
```

Αν τα permissions υπάρχουν μόνο:

</> Plain text

```
στο S3 bucket
στο Azure Storage
σε notebooks
σε clusters
σε BI tools
```

έχεις fragmented governance.

Το **Unity Catalog** προσπαθεί να κάνει το governance central.

Η Databricks το περιγράφει ως hierarchical governance layer στο οποίο κάθε governed asset είναι **securable object** και privileges μπορούν να δοθούν σε users, groups ή service principals. Το metastore είναι το top-level container και τα data assets οργανώνονται συνήθως στο three-level namespace `catalog.schema.object` . [🌐 Databricks Docu...](#) +1

2. Unity Catalog ≠ storage

Πολύ σημαντικό.

Το Unity Catalog δεν είναι:

</> Plain text

```
ένα τεράστιο database filesystem
```

Δεν είναι αυτό που αποθηκεύει τα bytes του Parquet.

Είναι περισσότερο:

</> Plain text

```
metadata
+
```

```
namespace
+
permissions
+
ownership
+
lineage
+
governance policies
+
storage references
```

Τα πραγματικά data μπορεί να είναι σε:

</> Plain text

```
S3
ADLS
GCS
```

Άρα:

</> Plain text

```
Unity Catalog
  |
  | governs
  ▼
table metadata
  |
  | points to / manages
  ▼
cloud storage
```

3. Το hierarchy από την κορυφή

Η βασική ιεραρχία είναι:

</> Plain text

```
METASTORE
|
├─ CATALOG: dev
|   ├── SCHEMA: bronze
|   ├── SCHEMA: silver
|   └─ SCHEMA: gold
|
├─ CATALOG: prod
|   ├── SCHEMA: bronze
|   ├── SCHEMA: silver
|   └─ SCHEMA: gold
|
└─ CATALOG: sandbox
```

και μέσα σε schema:

</> Plain text

```
prod
├── silver
│   ├── orders
│   ├── customers
│   ├── products
│   ├── files_volume
│   └── normalize_email()
```

Η Databricks θεωρεί το **catalog** primary unit data isolation και το schema ως πιο λεπτομερές organizational/access-control layer. [Databricks Docu...](#) +1

4. Metastore — τι είναι πραγματικά;

Το metastore είναι:

ο top-level container του Unity Catalog για metadata και permissions.

Δεν είναι:

</> Plain text

```
a table
a schema
a storage bucket
a workspace
```

Έχει metadata για:

</> Plain text

```
catalogs
tables
volumes
external locations
credentials
shares
permissions
...
```

και εκθέτει το three-level namespace:

</> Plain text

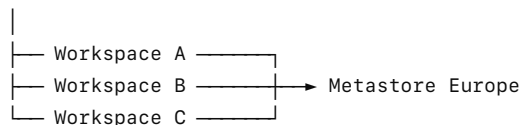
```
catalog.schema.object
```

Η Databricks αναφέρει ότι συνήθως υπάρχει **ένα Unity Catalog metastore ανά region**, και πολλά workspaces της ίδιας περιοχής μπορούν να συνδέονται σε αυτό. [Databricks Docu...](#) +1

Άρα:

</> Plain text

Databricks Account



Έτσι μπορείς να έχεις κοινό governance μεταξύ workspaces.

5. Workspace ≠ metastore

Θυμήσου από Phase 1:

</> Plain text

Workspace

=

operational development environment

Ενώ:

</> Plain text

Metastore

=

regional Unity Catalog governance container

Ένα notebook βρίσκεται σε workspace.

Ένα `prod.sales.orders` table είναι registered στο metastore.

Αυτό σημαίνει ότι δύο workspaces μπορούν να βλέπουν το ίδιο governed object, εφόσον:

</> Plain text

και τα δύο συνδέονται στο ίδιο metastore

+

ο principal έχει permission

+

δεν υπάρχει workspace binding που το απαγορεύει

6. Catalog

Το catalog είναι πρώτο επίπεδο του namespace:

</> Plain text

```
prod.sales.orders
  ^ ^ ^ ^
catalog
```

Σκέψου το ως μεγάλο logical/security boundary.

Παραδείγματα architecture:

</> Plain text

```
dev
test
prod
```

ή:

</> Plain text

```
finance
sales
marketing
```

ή:

</> Plain text

```
prod_finance
prod_sales
dev_shared
```

Δεν υπάρχει μία μοναδική σωστή δομή.

Η Databricks συνιστά τα catalogs να αντιπροσωπεύουν meaningful isolation boundaries, όπως environment, business unit, team ή project, και να χρησιμοποιούνται ώστε τα hierarchical grants να είναι απλά και λογικά. [Databricks Docu...](#) +1

7. Schema

Το schema είναι δεύτερο επίπεδο:

</> Plain text

```
prod.sales.orders
  ^ ^ ^ ^ ^
  schema
```

Schema μπορεί να οργανώνει objects κατά:

</> Plain text

```
layer
team
```

```
domain
use case
```

Για παράδειγμα:

</> Plain text

```
prod
├─ bronze
├─ silver
└─ gold
```

ή:

</> Plain text

```
prod
├─ finance
├─ logistics
└─ crm
```

Η Databricks ορίζει schema ως child του catalog που μπορεί να περιέχει tables, views, volumes, models και functions.

[Databricks Docu...](#)

8. Object

Το τρίτο όνομα:

</> Plain text

```
prod.sales.orders
      ^^^^^^
      object
```

Μπορεί να είναι:

</> Plain text

```
table
view
volume
function
model
materialized view
streaming table
...
```

Άρα ο γενικός όρος είναι:

</> Plain text

```
catalog.schema.object
```

όχι πάντα:

```
</> Plain text
```

```
catalog.schema.table
```

9. Fully qualified name

Αυτό:

```
</> SQL
```

```
SELECT *  
FROM prod.sales.orders;
```

είναι fully qualified.

Αντί:

```
</> SQL
```

```
SELECT *  
FROM orders;
```

Το δεύτερο εξαρτάται από current catalog/schema.

```
</> SQL
```

```
SELECT current_catalog();  
SELECT current_schema();
```

Μπορείς επίσης:

```
</> SQL
```

```
USE CATALOG prod;  
USE SCHEMA sales;
```

και μετά:

```
</> SQL
```

```
SELECT *  
FROM orders;
```

Σε production code, fully-qualified names συχνά είναι ασφαλέστερα γιατί μειώνουν ambiguity.

10. Δημιουργία catalog

Basic syntax:

```
</> SQL
```

```
CREATE CATALOG ecommerce;
```

και:

```
</> SQL
```

```
USE CATALOG ecommerce;
```

Για να δημιουργήσεις catalog χρειάζεσαι κατάλληλο privilege, τυπικά `CREATE CATALOG` στο metastore. [Databricks Docu...](#)

11. Δημιουργία schema

```
</> SQL
```

```
CREATE SCHEMA ecommerce.silver;
```

ή:

```
</> SQL
```

```
CREATE SCHEMA IF NOT EXISTS ecommerce.silver;
```

Για Unity Catalog schema creation χρειάζεσαι συνήθως:

```
</> Plain text
```

```
USE CATALOG  
+  
CREATE SCHEMA
```

στο parent catalog. [Databricks Docu...](#)

Αυτό είναι το πρώτο παράδειγμα ότι permissions είναι hierarchical.

12. Securable object

Αυτός είναι σημαντικός όρος.

Securable σημαίνει:

object στο οποίο μπορείς να δώσεις privileges.

Παράδειγμα:

Plain text

```
catalog
schema
table
volume
external location
storage credential
function
...
```

Η Databricks ορίζει securable object ακριβώς έτσι: object registered στο Unity Catalog πάνω στο οποίο grants μπορούν να δοθούν σε principal. [Databricks Docu...](#) +1

13. Principal

Principal = identity στην οποία δίνεις permissions.

Βασικά:

Plain text

```
user
group
service principal
```

Example:

Plain text

```
user:
nikos@company.com

group:
data_engineers

service principal:
orders-etl-prod
```

Η Databricks συνιστά να provisionάρεις principals στο **account level** και να χρησιμοποιείς groups αντί individual users όπου γίνεται. [Databricks Docu...](#)

14. Γιατί groups;

Αν έχεις:

Plain text

Nikos
Maria
John
Eva

μη γράφεις:

</> SQL

```
GRANT SELECT ... TO Nikos;  
GRANT SELECT ... TO Maria;  
GRANT SELECT ... TO John;  
GRANT SELECT ... TO Eva;
```

Καλύτερα:

</> Plain text

```
group = analysts
```

και:

</> SQL

```
GRANT SELECT  
ON TABLE prod.gold.daily_sales  
TO `analysts`;
```

Μετά membership management:

</> Plain text

```
Maria joins team  
→ add Maria to analysts
```

Δεν αλλάζεις 200 table grants.

15. Service Principal

Production automation:

</> Plain text

```
02:00 daily ETL
```

δεν πρέπει ιδανικά να εξαρτάται από:

</> Plain text

```
nikos@company.com
```

Αν φύγεις από εταιρεία:

</> Plain text

```
account disabled
```

δεν θέλεις να σταματήσει production.

Άρα:

</> Plain text

```
orders-etl-prod
```

ως service principal.

Η Databricks συνιστά production ownership και write access να ανατίθεται σε groups/service principals αντί προσωπικούς users. [Databricks Docu...](#)

16. Τι είναι privilege;

Privilege = επιτρεπόμενη action πάνω σε securable.

Example:

</> SQL

```
GRANT SELECT
ON TABLE ecommerce.silver.orders
TO `analysts`;
```

Parse:

</> Plain text

```
GRANT
δώσε

SELECT
permission

ON TABLE
σε τι object type;

ecommerce.silver.orders
ποιο object;

TO analysts
σε ποιο principal;
```

17. USE CATALOG

Αυτό είναι source σύγχυσης.

</> SQL

```
GRANT USE CATALOG
ON CATALOG ecommerce
TO `analysts`;
```

Δεν σημαίνει:

μπορούν να διαβάζουν όλα τα tables.

Σημαίνει:

μπορούν να traverse/use αυτό το catalog ως namespace, subject to additional object permissions.

Η Databricks λέει ρητά ότι `USE CATALOG` και `USE SCHEMA` είναι prerequisite access privileges, αλλά **δεν παρέχουν από μόνα τους `SELECT`**. [Databricks Docu...](#)

18. USE SCHEMA

</> SQL

```
GRANT USE SCHEMA
ON SCHEMA ecommerce.gold
TO `analysts`;
```

Πάλι δεν σημαίνει:

</> Plain text

```
SELECT everything
```

Χρειάζεσαι επίσης privilege στο object.

19. Για να διαβάσεις ένα table

Conceptually:

</> Plain text

```
USE CATALOG ecommerce
+
USE SCHEMA gold
+
SELECT daily_sales
```

SQL:

`</>` SQL

```
GRANT USE CATALOG
ON CATALOG ecommerce
TO `analysts`;

GRANT USE SCHEMA
ON SCHEMA ecommerce.gold
TO `analysts`;

GRANT SELECT
ON TABLE ecommerce.gold.daily_sales
TO `analysts`;
```

Τώρα μπορεί:

`</>` SQL

```
SELECT *
FROM ecommerce.gold.daily_sales;
```

20. Privilege inheritance

Εδώ είναι μια μεγάλη δύναμη.

Μπορείς να κάνεις:

`</>` SQL

```
GRANT SELECT
ON SCHEMA ecommerce.gold
TO `analysts`;
```

και το privilege μπορεί να κληρονομείται προς objects μέσα στο schema σύμφωνα με το Unity Catalog privilege inheritance model.

Ομοίως grants σε catalog μπορούν να ρέουν προς children. Η Databricks συνιστά να αξιοποιείς την hierarchy/inheritance ώστε να αποφεύγεις χιλιάδες per-table grants. [Databricks Docu...](#) +1

21. Άσκηση permission

Group:

`</>` Plain text

```
analysts
```

έχει:

</> Plain text

```
USE CATALOG ecommerce
USE SCHEMA ecommerce.gold
```

αλλά όχι `SELECT` .

Query:

</> SQL

```
SELECT *
FROM ecommerce.gold.daily_sales;
```

Τι γίνεται;

Λύση

Αποτυγχάνει.

Το `USE` δεν δίνει permission στα δεδομένα.

22. SELECT vs MODIFY

`SELECT` :

</> Plain text

```
read table
```

`MODIFY` :

</> Plain text

```
write/change table data
```

Το `MODIFY` historically καλύπτει broad DML capabilities.

Η current Unity Catalog έχει επίσης finer-grained DML privileges σε beta, όπως `INSERT` , ώστε να μπορείς να δίνεις πιο περιορισμένες write capabilities αντί broad `MODIFY` . [Databricks Docu...](#)

Για βασικό production model:

</> Plain text

```
analyst
→ SELECT

data engineer / pipeline SP
→ MODIFY
```

ανάλογα με το architecture.

23. Ownership

Κάθε securable έχει owner.

Owner έχει πολύ μεγαλύτερη δύναμη από απλό `SELECT` .

Μπορεί συνήθως:

</> Plain text

```
manage privileges
modify/manage object
transfer ownership
```

Η Databricks προτείνει **production objects να owned από groups**, όχι individual users. [Databricks Docu...](#)

Bad:

</> Plain text

```
prod.gold.revenue
owner = Nikos
```

Better:

</> Plain text

```
owner = data_platform_admins
```

24. MANAGE

`MANAGE` είναι privilege για delegated governance.

Principal με `MANAGE` μπορεί να διαχειρίζεται grants και ownership-related operations χωρίς να είναι απαραίτητα owner.

Σημαντικό:

</> Plain text

```
ALL PRIVILEGES
```

δεν περιλαμβάνει πάντα όλα όσα φαντάζεσαι. Για παράδειγμα η current Unity Catalog documentation αναφέρει ότι `MANAGE` , `EXTERNAL USE LOCATION` και `EXTERNAL USE SCHEMA` δεν συμπεριλαμβάνονται στο `ALL PRIVILEGES` . [Databricks Docu...](#)

Αυτό είναι καλό certification trap.

25. Show grants

Πολύ βασικό debugging:

</> SQL

```
SHOW GRANTS
ON TABLE ecommerce.gold.daily_sales;
```

ή:

</> SQL

```
SHOW GRANTS
ON SCHEMA ecommerce.gold;
```

ή:

</> SQL

```
SHOW GRANTS
ON CATALOG ecommerce;
```

Θέλεις πάντα να μπορείς να απαντήσεις:

</> Plain text

ποιος έχει τι;

26. Revoke

</> SQL

```
REVOKE SELECT
ON TABLE ecommerce.gold.daily_sales
FROM `analysts`;
```

Προσοχή όμως σε inherited grants.

Μπορεί να αφαιρέσεις direct table grant αλλά ο user να εξακολουθεί να έχει access επειδή υπάρχει:

</> Plain text

SELECT on schema

ή:

</> Plain text

```
SELECT on catalog
```

Άρα permission debugging πρέπει να εξετάζει και inheritance.

27. Managed vs external — πολύ σημαντικό

Τώρα πάμε στο storage.

Το Unity Catalog μπορεί να governάρει και:

```
</> Plain text
```

```
Managed assets
```

και:

```
</> Plain text
```

```
External assets
```

Η διαφορά δεν είναι:

```
</> Plain text
```

```
managed = inside Databricks servers
external = outside Databricks
```

Και τα δύο μπορεί να είναι στο **δικό σου cloud object storage**.

Η διαφορά είναι:

ποιος ελέγχει το lifecycle/location των underlying files.

Η Databricks λέει ότι managed assets έχουν Unity Catalog governance **και** storage lifecycle management, ενώ external assets έχουν Unity Catalog governance αλλά το storage lifecycle παραμένει δική σου/εξωτερικού συστήματος ευθύνη.

🌐 Databricks Docu...

28. Managed table

```
</> SQL
```

```
CREATE TABLE ecommerce.silver.orders (
  id BIGINT,
  amount DECIMAL(10,2)
);
```

χωρίς LOCATION .

Unity Catalog αποφασίζει storage path.

Conceptually:

Plain text

```
catalog/schema
managed storage root
  ↓
Unity-generated internal location
  ↓
Delta files
```

Αν κάνεις:

SQL

```
DROP TABLE ecommerce.silver.orders;
```

σε managed table, Unity Catalog διαχειρίζεται και deletion των underlying files σύμφωνα με lifecycle semantics.

 Databricks Docu...

29. External table

External:

SQL

```
CREATE TABLE ecommerce.silver.orders_ext
USING DELTA
LOCATION 's3://company-data/orders/';
```

ή Azure equivalent.

Εδώ:

Plain text

YOU specify storage location.

Αν:

SQL

```
DROP TABLE ecommerce.silver.orders_ext;
```

τότε:

Plain text

Unity Catalog metadata removed

αλλά:

</> Plain text

underlying files remain

Η Databricks documentation κάνει ακριβώς αυτή τη διάκριση. [Databricks Docu...](#)

30. Managed vs external — quick mental model

</> Plain text

MANAGED TABLE

UC:

metadata 

permissions 

lineage 

storage location 

storage lifecycle 

</> Plain text

EXTERNAL TABLE

UC:

metadata 

permissions 

lineage 

YOU:

storage location 

storage lifecycle 

31. Ποιο προτείνεται;

Για νέα Databricks-first workloads, η Databricks γενικά προτείνει managed tables.

External tables έχουν νόημα όταν:

</> Plain text

existing data already live at known paths

multiple non-Databricks systems control storage lifecycle

migration constraints

interoperability requirements

Managed tables δίνουν περισσότερο automation, optimization και governance integration. [Databricks Docu...](#) +1

32. Managed Storage Location

Πού αποθηκεύεται ένα managed table;

Unity Catalog μπορεί να έχει managed storage location σε:

</> Plain text

```
metastore
catalog
schema
```

και η πιο specific/lower-level διαθέσιμη location κερδίζει.

</> Plain text

```
schema managed location
  ↓ highest priority

else catalog managed location

else metastore managed location
```

Η Databricks πλέον συνιστά συχνά catalog-level managed storage ως isolation boundary. [Databricks Docu...](#) +1

33. Δημιουργία catalog με managed location

Conceptually:

</> SQL

```
CREATE CATALOG ecommerce
MANAGED LOCATION 's3://company-uc/ecommerce/';
```

Για να το κάνεις, η location πρέπει να βρίσκεται κάτω από Unity Catalog external location και χρειάζεσαι το κατάλληλο CREATE MANAGED STORAGE privilege. [Databricks Docu...](#)

Θα εξηγήσουμε σε λίγο τι σημαίνει external location.

34. Storage Credential

Τώρα πάμε βαθύτερα.

Έχεις cloud storage:

</> Plain text

```
s3://company-raw-data/
```

To Databricks χρειάζεται τρόπο να authenticate στο AWS.

Δεν θέλεις κάθε user να έχει:

</> Plain text

```
AWS access key
secret key
```

Άρα Unity Catalog έχει object:

storage credential

To storage credential abstractάρει long-term cloud identity/credential, π.χ.:

</> Plain text

```
AWS IAM role
Azure managed identity/service principal
GCP service account
```

ανά cloud.

Η Databricks ορίζει storage credential ως Unity Catalog object που abstractάρει long-term cloud credentials.

⊕ Databricks Docu... +1

35. External Location

Storage credential μόνο του λέει:

</> Plain text

```
"με ποια identity μπορώ να μιλήσω στο cloud;"
```

Δεν λέει:

</> Plain text

```
"ποιο path governάρουμε;"
```

To external location συνδυάζει:

</> Plain text

```
cloud path
+
storage credential
```

Conceptually:

</> Plain text

```
Storage Credential
  "IAM role X"
  |
  ▼
External Location
  path = s3://company/raw/
  credential = IAM role X
```

Η Databricks το ορίζει ρητά ως securable object που συνδυάζει URI και storage credential. [Databricks Docu...](#)

36. Το πλήρες chain

</> Plain text

```
AWS IAM Role / Azure Identity
  |
  ▼
Storage Credential
  |
  ▼
External Location
s3://company/raw/
  |
  ▼
External Table / External Volume
  |
  ▼
Files
```

Αυτό είναι ένα από τα πιο σημαντικά Unity Catalog diagrams.

37. Γιατί να μη δίνω storage credential απευθείας;

Γιατί external location είναι πιο granular.

Storage credential μπορεί να έχει πρόσβαση σε μεγάλο storage scope.

External location:

</> Plain text

```
s3://company/finance/
```

μπορεί να επιτρέπει access μόνο εκεί.

Η Databricks προτείνει, για example, `CREATE EXTERNAL TABLE` privileges να δίνονται στο external location αντί απευθείας στο storage credential όταν γίνεται, επειδή έτσι το permission scoped σε path. [Databricks Docu...](#)

38. External table lifecycle example

Cloud:

</> Plain text

```
s3://company/legacy/orders/
```

ήδη έχει Delta data.

Create:

</> SQL

```
CREATE TABLE ecommerce.silver.legacy_orders
USING DELTA
LOCATION 's3://company/legacy/orders/';
```

Απαραίτητη προϋπόθεση:

</> Plain text

```
location falls under governed external location
+
principal has required privileges
```

Τώρα Unity Catalog registerάρει metadata.

Δεν ξαναγράφει απαραίτητα τα data.

39. Volume — γιατί υπάρχει;

Τώρα κάτι που πολλοί αρχάριοι μπερδεύουν.

Table είναι για:

</> Plain text

```
tabular data
rows / columns
SQL/DataFrame semantics
```

Αλλά τι γίνεται με:

</> Plain text

```
PDF
JPEG
JSON config
ML files
checkpoints
text documents
```

```
binary artifacts
raw files
```

Δεν θέλεις να τα κάνεις όλα tables.

Εκεί έρχονται τα **Volumes**.

Volume = Unity Catalog governed storage object για non-tabular/file-based data.

40. Managed Volume

</> SQL

```
CREATE VOLUME ecommerce.raw.documents;
```

Unity Catalog επιλέγει managed storage location.

Files μπορούν να accessed από:

</> Plain text

```
/Volumes/ecommerce/raw/documents/...
```

41. External Volume

</> SQL

```
CREATE EXTERNAL VOLUME ecommerce.raw.documents_ext
LOCATION 's3://company/documents/';
```

Εδώ path είναι externally managed.

Όπως external table:

</> Plain text

```
drop volume
→ metadata removed

underlying files remain
```

για external volume. Managed volume drop μπορεί να remove underlying data as part of managed lifecycle. [Databricks Docu...](#)

42. Volume permissions

Tables:

⟨⟩ Plain text

```
SELECT
MODIFY
```

Volumes:

⟨⟩ Plain text

```
READ VOLUME
WRITE VOLUME
```

Example:

⟨⟩ SQL

```
GRANT READ VOLUME
ON VOLUME ecommerce.raw.documents
TO `data_scientists`;
```

Για file read χρειάζεσαι επίσης:

⟨⟩ Plain text

```
USE CATALOG
USE SCHEMA
READ VOLUME
```

Για writes:

⟨⟩ Plain text

```
USE CATALOG
USE SCHEMA
READ VOLUME
WRITE VOLUME
```

σύμφωνα με τη current volume privilege model. [Databricks Docu...](#)

43. File access

Python:

⟨⟩ Python

```
path = "/Volumes/ecommerce/raw/documents/sample.json"

with open(path, "r") as f:
    text = f.read()
```

ή Spark:

</> Python

```
df = spark.read.json(
    "/Volumes/ecommerce/raw/documents/events/"
)
```

Το σημαντικό:

</> Plain text

Volume gives governed file path semantics.

Η Databricks συνιστά στους περισσότερους users να χρησιμοποιούν table names για tables και `/Volumes/...` για direct non-tabular file access αντί να γράφουν raw cloud URIs παντού. [Databricks Docu...](#)

44. Tables vs Volumes

Need	Table	Volume
SQL rows/columns	✓	✗
DataFrame table semantics	✓	✗
PDF/images/files	✗	✓
SELECT *	✓	✗
READ VOLUME	✗	✓
Delta table history	✓	✗ as table semantics

Αυτό είναι εύκολο certification question.

45. Workspace Bindings

Έχεις δύο workspaces:

</> Plain text

```
dev-workspace
prod-workspace
```

και catalog:

</> Plain text

```
prod
```

Μπορείς να πεις:

το `prod catalog` μπορεί να χρησιμοποιηθεί μόνο από συγκεκριμένα `workspaces`.

Αυτό είναι **workspace binding**.

Άρα `permission` μπορεί να υπάρχει στον `user`, αλλά `access` να αποτυγχάνει επειδή το `current workspace` δεν έχει `binding` προς το `catalog`.

Η Unity Catalog access architecture χρησιμοποιεί `workspace bindings` ως ξεχωριστό `access-control mechanism` για `catalogs`, `external locations` και `storage credentials`. [Databricks Docu...](#) +1

46. Πολύ σημαντικό debugging implication

User έχει:

</> Plain text

```
USE CATALOG prod
USE SCHEMA prod.finance
SELECT prod.finance.payroll
```

και πάλι `failure`.

Γιατί;

Possible:

</> Plain text

```
prod catalog not bound to this workspace
```

Άρα:

</> Plain text

```
user privilege
≠
only possible authorization boundary
```

47. BROWSE

`BROWSE` είναι `interesting privilege`.

Μπορεί να επιτρέψει `discoverability/metadata visibility` χωρίς να σημαίνει `data access`.

Αυτό είναι χρήσιμο όταν θέλεις:

</> Plain text

```
users can discover that a dataset exists
```

αλλά πρέπει να requestάρουν access πριν κάνουν:

</> SQL

```
SELECT ...
```

Lineage visibility επίσης μπορεί να χρησιμοποιεί `BROWSE semantics` για object discovery. [Databricks Docu...](#)

48. Row-level security

Έστω table:

</> Plain text

```
sales

country | revenue
GR      | ...
DE      | ...
FR      | ...
```

Θέλεις:

</> Plain text

```
Greek team
→ only GR rows

German team
→ only DE rows
```

Μπορείς να χρησιμοποιήσεις **row filter**.

Η row filter είναι SQL UDF που εφαρμόζεται κατά query time και αποκλείει rows όπου επιστρέφει false. [Databricks Docu...](#)

Conceptually:

</> Plain text

```
SELECT *
FROM sales
  |
  ▼
row filter
  |
  └─ allowed → visible
```

└─ denied → removed

49. Simple row filter pattern

Suppose function:

</> SQL

```
CREATE FUNCTION security.country_filter(country STRING)
RETURN
CASE
  WHEN is_account_group_member('global_sales') THEN TRUE
  WHEN is_account_group_member('greece_sales') AND country = 'GR' THEN TRUE
  ELSE FALSE
END;
```

Then bind it conceptually to table:

</> SQL

```
ALTER TABLE ecommerce.gold.sales
SET ROW FILTER security.country_filter
ON (country);
```

Η ακριβής design production UDF πρέπει να γίνει carefully, αλλά αυτό είναι η βασική mechanism.

50. Column mask

Table:

</> Plain text

```
customer_id
name
email
salary
```

Analysts χρειάζονται rows αλλά όχι πραγματικό salary.

Column mask μπορεί να δίνει:

</> Plain text

```
manager:
salary = 5000

analyst:
salary = NULL / masked / bucketed
```

Η mask είναι query-time policy πάνω σε column value. [Databricks Docu...](#)

Conceptually:

</> Plain text

```
stored:
5000

query as finance_admin:
5000

query as analyst:
****
```

51. ABAC

Αν έχεις 5 tables, μπορείς να βάλεις row filter/column mask individually.

Αν έχεις:

</> Plain text

```
10,000 tables
```

αυτό δεν scaleάρει.

Εκεί έρχεται:

Attribute-Based Access Control (ABAC)

Μπορείς να χρησιμοποιήσεις governed tags.

Example:

</> Plain text

```
tag:
sensitivity = pii
```

και policy:

</> Plain text

```
if column tagged pii
→ mask it for users without privilege
```

Η Databricks πλέον προτείνει ABAC όταν θέλεις centralized tag-driven policies που εφαρμόζονται σε πολλά objects. ABAC μπορεί να εφαρμόσει row filters, column masks και πλέον policy-based GRANT/DENY capabilities σε supported scopes.

52. Table-level policy vs ABAC

</> Plain text

TABLE-LEVEL MASK

manually attach policy
to table A

manually attach
to table B

...

VS:

</> Plain text

ABAC

tag:
PII

policy:
mask PII

new tagged table tomorrow
→ automatically covered

Η Databricks συνιστά ABAC για consistent policy enforcement across many tables, ενώ direct row filters/masks έχουν νόημα για per-table logic. [🌐 Databricks Docu... +1](#)

53. Lineage

Έστω pipeline:

</> Plain text

```
bronze.orders
  ↓
silver.orders
  ↓
gold.daily_revenue
  ↓
dashboard
```

Unity Catalog μπορεί να captureάπει:

</> Plain text

upstream
downstream

relationships.

Η Databricks λέει ότι lineage συλλέγεται αυτόματα για queries που τρέχουν στο Databricks, μέχρι και **column-level lineage**, και συγκεντρώνεται μεταξύ workspaces που μοιράζονται το ίδιο metastore. [Databricks Docu...](#)

54. Γιατί lineage είναι πραγματικά χρήσιμο;

Έστω θέλεις να αλλάξεις:

</> Plain text

```
silver.orders.customer_id
```

Αν απλώς κάνεις:

</> SQL

```
ALTER ...
```

μπορεί να σπάσεις:

</> Plain text

```
gold.customer_360
dashboard
ML feature
report
```

Lineage:

</> Plain text

```
customer_id
  ↓
gold.customer_360
  ↓
executive_dashboard
```

σου επιτρέπει **impact analysis** πριν την αλλαγή. [Databricks Docu...](#)

55. Debugging με lineage

Dashboard έχει λάθος revenue.

Ξεκίνα:

</> Plain text

```
dashboard
```

```
↑
gold.daily_revenue
↑
silver.orders
↑
bronze.orders
```

και ρώτα:

</> Plain text

Σε ποιο layer εμφανίστηκε πρώτη φορά
η λανθασμένη τιμή;

Αυτό συνδέεται με τον debugging τρόπο που ήδη μάθαμε.

56. Unity Catalog UI

Στο Databricks workspace θα χρησιμοποιείς το **Catalog Explorer**.

Conceptually:

</> Plain text

```
Catalog Explorer
|
├── ecommerce
|   ├── bronze
|   │   └── orders_raw
|   ├── silver
|   │   └── orders
|   └── gold
|       └── daily_sales
|
```

Για κάθε object μπορείς να δεις ανάλογα με permission:

</> Plain text

```
schema
details
permissions
history
lineage
tags
dependencies
```

Μην το βλέπεις απλώς σαν file browser.

Είναι governance UI.

57. Production architecture example

Ας στήσουμε το e-commerce project.

Plain text

```
CATALOG
prod_ecommerce
|
|— bronze
|   |— orders_raw
|   |— customers_raw
|   └─ ingest_files [volume]
|
|— silver
|   |— orders
|   |— customers
|   └─ products
|
└─ gold
    |— daily_revenue
    |— customer_360
    └─ product_performance
```

Groups:

Plain text

```
data_engineers
analysts
finance_analysts
platform_admins
```

Service principals:

Plain text

```
orders_ingest_sp
silver_transform_sp
gold_pipeline_sp
```

58. Example grants

Engineer team:

SQL

```
GRANT USE CATALOG
ON CATALOG prod_ecommerce
TO `data_engineers`;

GRANT USE SCHEMA
ON SCHEMA prod_ecommerce.silver
TO `data_engineers`;
```

```
GRANT SELECT, MODIFY
ON SCHEMA prod_ecommerce.silver
TO `data_engineers`;
```

Production architecture may choose even tighter write access and reserve direct `MODIFY` for service principals, which Databricks explicitly recommends as a best practice. [🌐 Databricks Docu...](#)

59. Analyst grants

</> SQL

```
GRANT USE CATALOG
ON CATALOG prod_ecommerce
TO `analysts`;

GRANT USE SCHEMA
ON SCHEMA prod_ecommerce.gold
TO `analysts`;

GRANT SELECT
ON SCHEMA prod_ecommerce.gold
TO `analysts`;
```

Now analysts can query Gold.

But not Silver if no grants there.

60. Production service principal

</> SQL

```
GRANT USE CATALOG
ON CATALOG prod_ecommerce
TO `gold_pipeline_sp`;

GRANT USE SCHEMA
ON SCHEMA prod_ecommerce.silver
TO `gold_pipeline_sp`;

GRANT SELECT
ON SCHEMA prod_ecommerce.silver
TO `gold_pipeline_sp`;

GRANT USE SCHEMA
ON SCHEMA prod_ecommerce.gold
TO `gold_pipeline_sp`;

GRANT MODIFY
ON SCHEMA prod_ecommerce.gold
TO `gold_pipeline_sp`;
```

Conceptually:

Plain text

```
read Silver
write Gold
```

Least privilege.

61. Common bad practice

SQL

```
GRANT ALL PRIVILEGES
ON CATALOG prod_ecommerce
TO `all_company_users`;
```

Γιατί κακό;

Επειδή μπορεί να επιτρέψει πολύ περισσότερα από αυτά που πραγματικά χρειάζεται κάθε principal και μελλοντικά το `ALL PRIVILEGES` αξιολογείται δυναμικά καθώς προστίθενται applicable privileges. Η Databricks συνιστά restraint τόσο με `ALL PRIVILEGES` όσο και `MANAGE` . [Databricks Docu...](#) +1

62. Permission debugging — ο σωστός αλγόριθμος

Έστω error:

Plain text

```
PERMISSION_DENIED
```

για:

SQL

```
SELECT *
FROM prod_ecommerce.gold.daily_revenue;
```

Μη ζητήσεις αμέσως:

Plain text

```
"δώστε μου admin"
```

Debug με layers:

</> Plain text

1. Ποιος principal εκτελεί πραγματικά;
↓
2. Είναι σωστό workspace / catalog binding;
↓
3. USE CATALOG;
↓
4. USE SCHEMA;
↓
5. SELECT;
↓
6. row filter / column mask / ABAC;
↓
7. underlying external-storage permissions if relevant;
↓
8. compute compatible with Unity Catalog;

Αυτό είναι production-grade debugging model.

63. Case 1 — Wrong identity

Εσύ στο notebook:

</> Plain text

Nikos
SELECT 

Job:

</> Plain text

Run as:
orders-etl-sp

Job fails.

Γιατί;

orders-etl-sp :

</> Plain text

SELECT 

Το identity που πατάει "Run" δεν είναι απαραίτητα η identity που χρησιμοποιεί το workload.

64. Case 2 — SELECT υπάρχει, USE SCHEMA όχι

Grants:

</> Plain text

```
SELECT daily_revenue ✓  
USE CATALOG ✓  
USE SCHEMA ✗
```

Query:

</> SQL

```
SELECT *  
FROM prod_ecommerce.gold.daily_revenue;
```

μπορεί να αποτύχει επειδή δεν μπορείς να traverse το schema.

Η Databricks θεωρεί `USE CATALOG + USE SCHEMA` prerequisites για data access. [Databricks Docu...](#)

65. Case 3 — direct grant revoked but access remains

User είχε:

</> Plain text

```
SELECT TABLE daily_revenue
```

Admin κάνει:

</> SQL

```
REVOKE SELECT  
ON TABLE prod_ecommerce.gold.daily_revenue  
FROM `analysts`;
```

User ακόμα διαβάζει.

Γιατί;

Πιθανό:

</> Plain text

```
SELECT ON SCHEMA gold
```

ή catalog-level inherited grant.

Always inspect hierarchy.

66. Case 4 — managed table accessible, external fails

Managed table:

</> Plain text

```
works
```

External table:

</> Plain text

```
PERMISSION_DENIED / storage access error
```

Possible cause:

</> Plain text

```
external location
storage credential
workspace binding
cloud IAM
```

Αρα external storage προσθέτει extra layers.

67. Case 5 — volume file inaccessible

User:

</> Plain text

```
USE CATALOG 
USE SCHEMA 
```

but:

</> Python

```
open("/Volumes/.../file.json")
```

fails.

Check:

</> Plain text

```
READ VOLUME
```

Για writes:

</> Plain text

WRITE VOLUME

επίσης. [Databricks Docu...](#)

68. Case 6 — user sees masked salary

User λέει:

Έχω `SELECT` , γιατί βλέπω `****` ;

Επειδή:

</> Plain text

```
SELECT
#
unfiltered raw visibility
```

Fine-grained policies μπορούν να αλλάξουν row/column visibility.

Check:

</> Plain text

```
column mask
row filter
ABAC
```

69. Managed table drop vs external table drop

Certification classic:

</> SQL

```
DROP TABLE managed_table;
```

Unity Catalog handles metadata and underlying file lifecycle.

</> SQL

```
DROP TABLE external_table;
```

Unity Catalog removes metadata, but external files remain. [Databricks Docu...](#)

Αυτό πρέπει να το ξέρεις άνετα.

70. Storage Credential vs External Location

Άλλο certification classic:

</> Plain text

```
Storage Credential
=
how Databricks authenticates to cloud storage

External Location
=
which cloud path + which credential
```

Ένας credential μπορεί conceptually να χρησιμοποιηθεί από πολλά external locations:

</> Plain text

```
credential A
|
├─ s3://company/raw/
├─ s3://company/archive/
└─ s3://company/shared/
```

με ξεχωριστά Unity Catalog access boundaries.

71. External Location vs External Table

</> Plain text

```
External Location
=
governed storage path

External Table
=
table metadata pointing at tabular data
inside governed path
```

Δεν είναι συνώνυμα.

72. Metastore vs Catalog

</> Plain text

```
Metastore
=
top regional governance container

Catalog
=
```

```
primary logical/data isolation container
inside metastore
```

Δεν δημιουργείς ένα metastore για κάθε project ως default pattern.

Η Databricks συνιστά data isolation να ξεκινά συχνά στο catalog level. [Databricks Docu...](#)

73. Unity Catalog compute requirement

Δεν αρκεί να έχεις permissions.

To compute πρέπει να είναι Unity Catalog-compatible.

Η Databricks αναφέρει ότι classic compute με Standard ή Dedicated access modes μπορεί να accessάρει Unity Catalog, ενώ serverless και Databricks SQL compute το υποστηρίζουν. [Databricks Docu...](#)

Άρα:

</> Plain text

```
permissions correct
+
wrong legacy compute
=
still failure
```

74. Lakehouse Federation — πού μπαίνει;

Μερικές φορές δεν θέλεις να αντιγράψεις data στο Databricks.

Έχεις external database:

</> Plain text

```
PostgreSQL
Snowflake
SQL Server
...
```

και θέλεις να queryάρεις τα data μέσω Unity Catalog governance.

Lakehouse Federation χρησιμοποιεί:

</> Plain text

```
connection
↓
foreign catalog
↓
external database metadata
```

Δεν θα μπορούμε τώρα βαθιά στο connector-specific syntax, αλλά conceptually:

Plain text

```
Unity Catalog
  |
  └─ Foreign Catalog
      ↓
    external database
```

Χρήσιμο για query federation χωρίς full ingestion.

To CREATE CONNECTION / CREATE FOREIGN CATALOG privilege model είναι μέρος της Unity Catalog privileges architecture.

[Databricks Docu...](#)

75. Delta Sharing

Αντίθετο-ish use case:

Θέλω να μοιραστώ governed data με άλλο organization / recipient.

Delta Sharing είναι open sharing protocol integrated με Unity Catalog.

Conceptually:

Plain text

```
your UC tables
  ↓
share
  ↓
recipient
```

χωρίς να αντιγράψεις απαραίτητα manually files.

Θα το δούμε βαθύτερα όταν φτάσουμε cross-organization sharing, αλλά στο mental model:

Plain text

```
Federation
→ query data where it lives elsewhere

Delta Sharing
→ safely expose/share data outward
```

76. Full mini-project configuration

Για το project μας:

Plain text

```
Metastore:
Europe

Catalog:
prod_ecommerce

Managed storage:
cloud://company-uc/prod-ecommerce/

Schemas:
bronze
silver
gold
raw_files

Tables:
prod_ecommerce.bronze.orders_raw
prod_ecommerce.silver.orders
prod_ecommerce.gold.daily_revenue

Volume:
prod_ecommerce.raw_files.incoming

Service principals:
orders_ingest_sp
transform_sp
```

77. Practical SQL lab

Ξεκίνα:

</> SQL

```
CREATE CATALOG IF NOT EXISTS ecommerce_dev;
```

Create schemas:

</> SQL

```
CREATE SCHEMA IF NOT EXISTS ecommerce_dev.bronze;
CREATE SCHEMA IF NOT EXISTS ecommerce_dev.silver;
CREATE SCHEMA IF NOT EXISTS ecommerce_dev.gold;
CREATE SCHEMA IF NOT EXISTS ecommerce_dev.files;
```

Create table:

</> SQL

```
CREATE TABLE ecommerce_dev.silver.orders (
  order_id BIGINT,
  customer_id BIGINT,
  amount DECIMAL(10,2),
  country STRING
);
```

Create volume:

</> SQL

```
CREATE VOLUME ecommerce_dev.files.incoming;
```

78. Inspect objects

</> SQL

```
SHOW CATALOGS;
```

</> SQL

```
SHOW SCHEMAS IN ecommerce_dev;
```

</> SQL

```
SHOW TABLES IN ecommerce_dev.silver;
```

</> SQL

```
DESCRIBE TABLE ecommerce_dev.silver.orders;
```

This is basic UC navigation.

79. Grant practice

Assume group:

</> Plain text

```
training_analysts
```

Grant namespace:

</> SQL

```
GRANT USE CATALOG
ON CATALOG ecommerce_dev
TO `training_analysts`;
```

```
GRANT USE SCHEMA
ON SCHEMA ecommerce_dev.gold
TO `training_analysts`;
```

Then read:

```
</> SQL
```

```
GRANT SELECT
ON SCHEMA ecommerce_dev.gold
TO `training_analysts`;
```

Inspect:

```
</> SQL
```

```
SHOW GRANTS
ON SCHEMA ecommerce_dev.gold;
```

80. Volume practice

```
</> SQL
```

```
GRANT USE SCHEMA
ON SCHEMA ecommerce_dev.files
TO `data_engineers`;

GRANT READ VOLUME, WRITE VOLUME
ON VOLUME ecommerce_dev.files.incoming
TO `data_engineers`;
```

Then from Python:

```
</> Python
```

```
path = "/Volumes/ecommerce_dev/files/incoming/test.txt"

with open(path, "w") as f:
    f.write("hello unity catalog")
```

and:

```
</> Python
```

```
with open(path, "r") as f:
    print(f.read())
```

81. Real interview question

"What is the difference between a managed table and an external table in Unity Catalog?"

Strong answer:

Both are governed by Unity Catalog for metadata, access control, auditing, and lineage. For a managed table, Unity Catalog

additionally controls the underlying storage location and file lifecycle. For an external table, I provide the storage path and the underlying files remain externally managed; dropping the external table removes the catalog metadata but does not delete the underlying files.

That distinction is exactly how current Databricks docs define managed versus external assets. [Databricks Docu...](#)

82. Interview: Storage credential vs external location

A storage credential represents the cloud identity Databricks can use to authenticate to object storage. An external location binds that credential to a specific governed cloud-storage URI, giving Unity Catalog a securable path boundary on which privileges can be granted. [Databricks Docu...](#) +1

83. Interview: USE SCHEMA vs SELECT

`USE SCHEMA` allows a principal to traverse/use the schema namespace, but it does not grant access to table data. To query a table, the user typically also needs `USE CATALOG` on the parent catalog and `SELECT` on the table or an inherited parent grant. [Databricks Docu...](#)

84. Interview: volume vs table

A table provides structured relational semantics over rows and columns, while a Unity Catalog volume provides governed access to non-tabular files in cloud storage. Tables use privileges such as `SELECT` and `MODIFY`, whereas volumes use `READ VOLUME` and `WRITE VOLUME`. [Databricks Docu...](#)

85. Big test — Phase 3

Q1

What is the top-level Unity Catalog object?

Answer: Metastore.

Q2

What does:

</> Plain text

```
prod.finance.payroll
```

mean?

Answer:

Plain text

```
catalog = prod
schema = finance
object = payroll
```

Q3

Is workspace above catalog in the Unity Catalog namespace?

Answer: No. Workspace is operational environment; catalog is inside a Unity Catalog metastore.

Q4

Can multiple workspaces use one metastore?

Answer: Yes, typically within the same regional metastore architecture, subject to workspace bindings and permissions.

 Databricks Docu... +1

Q5

What is a securable object?

Answer: A Unity Catalog object on which privileges can be granted to principals.

Q6

What is a principal?

Answer: A user, group, or service principal receiving permissions.

Q7

Does `USE CATALOG` let me `SELECT` every table?

Answer: No.

Q8

Typical read path?

Plain text

```
USE CATALOG
+
USE SCHEMA
+
```

SELECT

Q9

Why are groups preferred to individual grants?

Answer: They centralize membership management and reduce permission sprawl.

Q10

Why use service principals for production pipelines?

Answer: Stable machine identity, decoupled from individual employee lifecycle.

Q11

What is ownership?

Answer: Administrative/control relationship with an object, including broad object management and grant delegation capabilities.

Q12

Is `ALL PRIVILEGES` literally every possible privilege?

Answer: No. For example `MANAGE` , `EXTERNAL USE LOCATION` and `EXTERNAL USE SCHEMA` are excluded. [Databricks Docu...](#)

Q13

Managed vs external table?

Answer:

`</>` Plain text

managed:

UC controls governance + storage lifecycle

external:

UC controls governance

you/external system controls storage lifecycle

Q14

Dropping external table deletes files?

Answer: No. Underlying files remain. [Databricks Docu...](#)

Q15

Dropping managed table?

Answer: Unity Catalog manages deletion/lifecycle of the underlying files. [Databricks Docu...](#)

Q16

Storage credential?

Answer: Governed abstraction of cloud authentication identity.

Q17

External location?

Answer: Cloud path + storage credential, represented as a securable Unity Catalog object.

Q18

Why external location instead of granting raw credential?

Answer: Better path-scoped least privilege.

Q19

What is a volume?

Answer: Unity Catalog governed container for non-tabular files.

Q20

Volume read privileges?

Answer:

</> Plain text

```
USE CATALOG
USE SCHEMA
READ VOLUME
``` :chatgpt-content-reference{index="52"}
```

---

### Q21

Table permissions and volume permissions are identical?

**\*\*Answer:\*\*** No.

Tables use e.g. `SELECT`, `MODIFY`; volumes use `READ VOLUME`, `WRITE VOLUME`.

---

### Q22

What is workspace binding?

**\*\*Answer:\*\*** Restriction specifying which workspaces can access certain UC objects such as catalogs, external locations, or

---

## Q28

Does Unity Catalog capture column-level lineage?

**Answer:** Yes, for supported Databricks queries/workloads. [Databricks Docu...](#)

## Q29

What is a likely debugging sequence for:

</> Plain text

PERMISSION\_DENIED

**Answer:**

</> Plain text

actual principal

↓

workspace binding

↓

USE CATALOG

↓

USE SCHEMA

↓

object privilege

↓

fine-grained policies

↓

storage/external location

↓

compute compatibility

## Q30

Why can a job fail although your notebook succeeds?

**Answer:** Job may execute as a different service principal with different Unity Catalog privileges.

## Phase 3 production exercise

Για πραγματική εξάσκηση, χτίσε νοητικά ή σε sandbox:

</> Plain text

```
ecommerce_dev
|
├─ bronze
├─ silver
├─ gold
└─ files
```

και κάνε όλη τη ροή:

</> SQL

```
CREATE CATALOG
CREATE SCHEMA
CREATE TABLE
CREATE VOLUME

GRANT USE CATALOG
GRANT USE SCHEMA
GRANT SELECT
GRANT MODIFY
GRANT READ VOLUME
GRANT WRITE VOLUME

SHOW GRANTS
REVOKE
```

Μετά δημιούργησε δύο roles νοητικά:

</> Plain text

```
analyst
pipeline service principal
```

και απάντησε ποιος πρέπει να μπορεί να:

</> Plain text

```
read Gold
write Gold
read Silver
create table
write volume
manage grants
```

Το σημαντικότερο outcome της Phase 3 είναι να μη σκέφτεσαι πλέον ένα permission error ως «δεν έχω SELECT». Να σκέφτεσαι **ολόκληρη την authorization chain**.

Σύμφωνα με το roadmap σου, το **Phase 4 είναι Data Ingestion**: explicit schemas vs inference, `spark.read` , CSV/JSON/Parquet, `COPY INTO` , και μετά πολύ βαθιά **Auto Loader** με `cloudFiles` , `file discovery`, `schemaLocation` , checkpoints, schema evolution, rescued data και duplicate/missing-file debugging. [databricks\\_mastery\\_phased\\_plan](#)

[Continue to next phase](#)

Worked for 7s

## Phase 4 — Data Ingestion

Συνεχίζουμε ακριβώς σύμφωνα με το roadmap σου: η **Phase 4** έχει στόχο να μάθεις πώς εισάγουμε δεδομένα σωστά, incrementally και reproducibly, ξεκινώντας από απλά file reads και φτάνοντας σε `COPY INTO` και **Auto Loader** με `cloudFiles` , checkpoints, `schemaLocation` , schema evolution, rescued data και debugging missing/duplicate files.

[databricks\\_mastery\\_phased\\_plan](#)

Η φάση αυτή είναι κρίσιμη γιατί σχεδόν κάθε data pipeline ξεκινά από:

`</>` Plain text

```
κάπου υπάρχουν δεδομένα
 ↓
πρέπει να τα ανακαλύψω
 ↓
πρέπει να τα διαβάσω
 ↓
πρέπει να τα ερμηνεύσω με schema
 ↓
πρέπει να θυμάμαι τι έχω ήδη επεξεργαστεί
 ↓
πρέπει να τα γράψω αξιόπιστα σε Delta
```

Θα προχωρήσουμε από το χαμηλότερο abstraction:

`</>` Plain text

```
FILES
 ↓
spark.read
 ↓
schema
 ↓
batch ingestion
 ↓
COPY INTO
 ↓
Auto Loader
 ↓
incremental discovery
 ↓
checkpoint + schemaLocation
 ↓
schema evolution
 ↓
rescued data
```



production debugging

## 1. Τι σημαίνει ingestion;

**Ingestion** σημαίνει:

η διαδικασία με την οποία δεδομένα που βρίσκονται έξω από το target data platform εισάγονται σε αυτό.

Παράδειγμα:

</> Plain text

External system

|

└─ CSV files

└─ JSON

└─ Parquet

└─ API export

└─ Kafka

└─ Database

└─ object storage



ingestion



Databricks / Delta table

Στη Phase 4 επικεντρωνόμαστε κυρίως σε:

**file-based ingestion**

δηλαδή data που φτάνουν ως files.

## 2. Ένα πραγματικό scenario

Έστω e-commerce εταιρεία.

Κάθε 5 λεπτά κάποιο operational system γράφει:

</> Plain text

s3://company/orders/

με files:

</> Plain text

orders\_2026\_09\_28\_0001.json

orders\_2026\_09\_28\_0002.json

orders\_2026\_09\_28\_0003.json

...

Θέλουμε:

</> Plain text

new files

↓

Bronze

↓

Silver

↓

Gold

Το πρώτο πρόβλημα είναι:

πώς διαβάζω αυτά τα files;

### 3. spark.read

Η πιο βασική PySpark ingestion API:

</> Python

```
df = spark.read.format("json").load(path)
```

Αυτό είναι **batch read**.

Meaning:

</> Plain text

path currently contains files

↓

Spark enumerates input

↓

reads them

↓

returns DataFrame

Παράδειγμα:

</> Python

```
orders = (
 spark.read
 .format("json")
 .load("/Volumes/ecommerce/raw/incoming/orders/")
)
```

ή συντομότερα:

```
</> Python
```

```
orders = spark.read.json(
 "/Volumes/ecommerce/raw/incoming/orders/"
)
```

## 4. CSV

CSV χρειάζεται parsing options.

```
</> Python
```

```
df = (
 spark.read
 .format("csv")
 .option("header", "true")
 .load(path)
)
```

Example file:

```
</> csv
```

```
order_id,customer_id,amount
1001,17,25.90
1002,18,10.50
```

Αν:

```
</> Python
```

```
.option("header", "true")
```

τότε Spark χρησιμοποιεί την πρώτη row ως column names.

Αν δεν το βάλεις:

```
</> Plain text
```

```
_c0
_c1
_c2
```

## 5. Delimiter

CSV δεν σημαίνει πάντα comma.

File:

</> Plain text

```
1001|17|25.90
1002|18|10.50
```

Χρειάζεσαι:

</> Python

```
df = (
 spark.read
 .format("csv")
 .option("delimiter", "|")
 .load(path)
)
```

## 6. Common CSV options

Θα συναντήσεις συχνά:

</> Python

```
.option("header", "true")
.option("delimiter", ",")
.option("quote", "'')
.option("escape", "'')
.option("multiline", "true")
.option("nullValue", "NULL")
```

Δεν πρέπει να αποστηθίσεις όλες τις options.

Θέλω να μάθεις να σκέφτεσαι:

το file format είναι grammar. Πρέπει να πω στον parser πώς είναι δομημένο.

## 7. Schema inference

Αν δεν δώσεις schema, Spark μπορεί να inferάρει.

Παράδειγμα:

</> Python

```
df = (
 spark.read
 .option("header", "true")
 .option("inferSchema", "true")
 .csv(path)
)
```

Input:

</> Plain text

```
id amount
1 15.2
2 20.1
```

Spark μπορεί να infer:

</> Plain text

```
id INT
amount DOUBLE
```

## 8. Γιατί schema inference έχει κόστος;

Για να ξέρει αν:

</> Plain text

"123"

είναι:

</> Plain text

```
INT
STRING
LONG
```

πρέπει να δει data.

Άρα schema inference σημαίνει:

</> Plain text

```
inspect input
↓
decide types
↓
then read actual data
```

Η Databricks documentation λέει ρητά ότι όταν δίνεις explicit schema, μπορεί να παραλειφθεί το schema-inference pass και έτσι να επιταχυνθεί η ανάγνωση. [Databricks Docu...](#)

## 9. Explicit schema

Production pipeline:

**</> Python**

```
from pyspark.sql.types import (
 StructType,
 StructField,
 LongType,
 DoubleType,
 StringType,
 TimestampType,
)

schema = StructType([
 StructField("order_id", LongType(), True),
 StructField("customer_id", LongType(), True),
 StructField("amount", DoubleType(), True),
 StructField("status", StringType(), True),
 StructField("order_ts", TimestampType(), True),
])
```

και:

**</> Python**

```
df = (
 spark.read
 .schema(schema)
 .json(path)
)
```

Τώρα λες:

Δεν θέλω να μαντέψεις. Αυτό είναι το contract.

## 10. DDL schema syntax

Δεν χρειάζεται πάντα StructType .

Μπορείς:

**</> Python**

```
schema = """
order_id BIGINT,
customer_id BIGINT,
amount DOUBLE,
status STRING,
order_ts TIMESTAMP
"""
```

και:

**</> Python**

```
df = (
 spark.read
 .schema(schema)
 .json(path)
)
```

Η Databricks υποστηρίζει schema είτε ως `StructType` είτε ως DDL-formatted string. [Databricks Docu...](#)

Για readability, το DDL string είναι συχνά πολύ βολικό.

## 11. Explicit schema vs inference

Για exploratory work:

Plain text

```
infer schema
```

είναι convenient.

Για production ingestion:

Plain text

```
explicit schema
```

είναι συχνά ασφαλέστερο γιατί:

Plain text

```
faster
predictable
version-controlled
no accidental type surprises
easier validation
```

## 12. Schema inference bug example

File 1:

Plain text

```
amount
100
200
```

Spark infer:

```
</> Plain text
```

```
INT
```

Αύριο file:

```
</> Plain text
```

```
amount
100.50
```

Τώρα:

```
</> Plain text
```

```
type changed
```

ή parsing issues.

Αυτό είναι ένας λόγος που ingestion pipelines πρέπει να έχουν deliberate schema policy.

---

## 13. JSON

JSON:

```
</> JSON
```

```
{"order_id": 1, "customer": "Maria", "amount": 20.5}
{"order_id": 2, "customer": "John", "amount": 50.0}
```

Read:

```
</> Python
```

```
df = spark.read.json(path)
```

ή:

```
</> Python
```

```
df = (
 spark.read
 .schema(schema)
 .json(path)
)
```

JSON μπορεί να έχει nested structure:

```
</> JSON
```

```
{
 "order_id": 1,
 "customer": {
 "id": 100,
 "name": "Maria"
 }
}
```

Spark schema:

```
</> Plain text
```

```
order_id: long
customer: struct
 id: long
 name: string
```

## 14. Parquet

Parquet είναι typed.

```
</> Python
```

```
df = spark.read.parquet(path)
```

Parquet files ήδη αποθηκεύουν schema metadata.

Άρα δεν χρειάζεται το ίδιο style parsing όπως CSV.

## 15. CSV vs JSON vs Parquet

Mental model:

```
</> Plain text
```

```
CSV
→ textual
→ weak/no native types

JSON
→ semi-structured
→ types inferred from content

Parquet
→ columnar
→ schema encoded in file
```

Για αυτό production lakehouses συχνά ingest raw CSV/JSON και μετά γράφουν Delta/Parquet-backed tables.

## 16. read\_files() — modern Databricks SQL ingestion helper

Databricks έχει και SQL table-valued function:

</> SQL

```
SELECT *
FROM read_files(
 '/Volumes/ecommerce/raw/orders/',
 format => 'json'
);
```

Το `read_files` μπορεί να διαβάσει CSV, JSON, XML, text, binary, Parquet, Avro και ORC, και μπορεί να inferάει `format/schema`. [Databricks Docu...](#)

Μπορείς explicit:

</> SQL

```
SELECT *
FROM read_files(
 '/Volumes/ecommerce/raw/orders/',
 format => 'csv',
 schema => 'order_id BIGINT, customer_id BIGINT, amount DOUBLE'
);
```

## 17. Πού χρησιμοποιείται read\_files ;

Πολύ χρήσιμο σε:

</> Plain text

Databricks SQL  
Lakeflow pipelines  
ad-hoc file exploration

και ειδικά θα το ξαναδούμε στη Phase 6, γιατί όταν χρησιμοποιείται σε streaming table με `STREAM`, το Databricks χρησιμοποιεί Auto Loader από κάτω. [Databricks Docu...](#)

## 18. Metadata columns

Όταν ingestάρεις files, συχνά θες να ξέρεις:

</> Plain text

από ποιο file ήρθε το row;  
πότε δημιουργήθηκε το file;  
ποιο source το έστειλε;

Η πληροφορία source file είναι πολύ χρήσιμη για debugging/auditing.

Conceptually:

⟨/⟩ Plain text

```
business data
+
ingestion metadata
```

Example Bronze:

⟨/⟩ Plain text

```
order_id
amount
source_file
ingestion_ts
```

Αυτό βοηθά όταν κάποιος ρωτήσει:

Από ποιο source file ήρθε αυτή η κακή εγγραφή;

## 19. Το πρώτο μεγάλο πρόβλημα με `plain spark.read`

Έστω folder:

⟨/⟩ Plain text

```
orders/
├─ file1.json
├─ file2.json
```

Τρέχεις:

⟨/⟩ Python

```
spark.read.json(path)
```

και γράφεις Bronze.

Αύριο folder:

⟨/⟩ Plain text

```
orders/
├─ file1.json
├─ file2.json
├─ file3.json
```

Αν ξανατρέξεις:

```
</> Python
```

```
spark.read.json(path)
```

τι διαβάζει;

Όλα τα files.

Άρα:

```
</> Plain text
```

```
file1
file2
file3
```

και όχι μόνο:

```
</> Plain text
```

```
file3
```

Αυτό είναι πρόβλημα για incremental ingestion.

---

## 20. Naive solution: filter by filename/date

Μπορείς να πεις:

```
</> Python
```

```
path = "/orders/2026-09-28/*.json"
```

Αυτό λειτουργεί μόνο αν upstream storage layout είναι reliable.

Αλλά έχει προβλήματα:

```
</> Plain text
```

```
late files
backfills
reuploads
timezone bugs
renamed files
missing expected folders
```

Άρα χρειαζόμαστε πιο robust ingestion semantics.

## 21. COPY INTO

Το πρώτο σημαντικό Databricks ingestion primitive.

Syntax:

</> SQL

```
COPY INTO ecommerce.bronze.orders_raw
FROM '/Volumes/ecommerce/raw/orders/'
FILEFORMAT = JSON;
```

Το `COPY INTO` φορτώνει files σε Delta table και είναι **retryable και idempotent**: files που έχουν ήδη φορτωθεί skipάρονται σε επόμενα runs. [Databricks Docu...](#) +1

## 22. Τι σημαίνει idempotent εδώ;

Folder:

</> Plain text

```
file1.json
file2.json
```

Run 1:

</> Plain text

```
COPY INTO
→ loads file1
→ loads file2
```

Run 2:

</> Plain text

```
COPY INTO
→ sees file1 already loaded
→ skip
→ sees file2 already loaded
→ skip
```

New file:

</> Plain text

```
file3.json
```

Run 3:

</> Plain text

```
skip file1
skip file2
load file3
```

Αυτό είναι πολύ πιο ασφαλές από naive:

</> Plain text

```
read entire folder every run
```

## 23. Important subtlety

Η Databricks λέει:

αν ένα file έχει ήδη φορτωθεί, το `COPY INTO` το skipάρει σε επόμενα runs, ακόμα κι αν το file έχει αλλάξει.

 Databricks Docu...

Άρα:

</> Plain text

```
same filename / same tracked file identity
```

δεν είναι απλά:

ξαναφόρτωσέ το επειδή το content άλλαξε.

Αυτό είναι σημαντικό σε debugging reuploads.

## 24. COPY INTO target

`COPY INTO` γράφει σε Delta table.

Example:

</> SQL

```
CREATE TABLE IF NOT EXISTS ecommerce.bronze.orders_raw (
 order_id BIGINT,
 customer_id BIGINT,
 amount DOUBLE,
 status STRING
);
```

then:

</> SQL

```
COPY INTO ecommerce.bronze.orders_raw
FROM '/Volumes/ecommerce/raw/orders/'
FILEFORMAT = JSON;
```

Η target table πρέπει να είναι Delta. [Databricks Docu...](#)

## 25. CSV COPY INTO

**</> SQL**

```
COPY INTO ecommerce.bronze.orders_raw
FROM '/Volumes/ecommerce/raw/orders_csv/'
FILEFORMAT = CSV
FORMAT_OPTIONS (
 'header' = 'true',
 'delimiter' = ','
);
```

## 26. PATTERN

Folder:

**</> Plain text**

```
incoming/
├─ orders_001.csv
├─ orders_002.csv
└─ customers_001.csv
```

Θέλεις μόνο orders:

**</> SQL**

```
COPY INTO ecommerce.bronze.orders_raw
FROM '/Volumes/ecommerce/raw/incoming/'
FILEFORMAT = CSV
PATTERN = 'orders_.*[.]csv'
FORMAT_OPTIONS ('header' = 'true');
```

PATTERN επιτρέπει filtering source files. [Databricks Docu...](#)

## 27. FILES

Μπορείς να φορτώσεις explicit files:

**</> SQL**

```
COPY INTO target
```

```
FROM '/path/'
FILEFORMAT = JSON
FILES = (
 'file1.json',
 'file2.json'
);
```

Current FILES limit είναι έως 1000 filenames ανά statement. [Databricks Docu...](#)

## 28. VALIDATE

Πολύ χρήσιμο.

Θέλεις να ελέγξεις ότι τα files:

</> Plain text

```
parse correctly
match schema
respect constraints
```

χωρίς να γράψεις data.

</> SQL

```
COPY INTO ecommerce.bronze.orders_raw
FROM '/Volumes/ecommerce/raw/orders/'
FILEFORMAT = JSON
VALIDATE ALL;
```

Το VALIDATE ελέγχει parsing, schema compatibility/evolution και nullability/check constraints χωρίς να κάνει write.

[Databricks Docu...](#)

Αυτό είναι εξαιρετικό pre-production/debugging tool.

## 29. COPY INTO transformation

Μπορείς να κάνεις transformation:

</> SQL

```
COPY INTO ecommerce.bronze.orders_raw
FROM (
 SELECT
 order_id::BIGINT,
 customer_id::BIGINT,
 amount::DOUBLE,
 upper(status) AS status
 FROM '/Volumes/ecommerce/raw/orders/'
)
FILEFORMAT = JSON;
```

Άρα `COPY INTO` δεν είναι απλώς byte copy.

Μπορεί να κάνει parsing/projection/casts.

## 30. COPY INTO schema inference

Μπορείς:

`</>` SQL

```
COPY INTO ecommerce.bronze.orders_raw
FROM '/path/'
FILEFORMAT = CSV
FORMAT_OPTIONS (
 'inferSchema' = 'true'
);
```

Για schema evolution:

`</>` SQL

```
COPY INTO ecommerce.bronze.orders_raw
FROM '/path/'
FILEFORMAT = CSV
FORMAT_OPTIONS (
 'inferSchema' = 'true',
 'mergeSchema' = 'true'
)
COPY_OPTIONS (
 'mergeSchema' = 'true'
);
```

Η Databricks διαχωρίζει:

`</>` Plain text

```
FORMAT_OPTIONS mergeSchema
→ merge source-file schemas
```

```
COPY_OPTIONS mergeSchema
→ evolve target Delta schema
```

Αυτό είναι σημαντική διάκριση. [Databricks Docu...](#) +1

## 31. COPY INTO vs Auto Loader

Εδώ αρχίζει το σημαντικό architecture decision.

Η Databricks λέει:

⟨/⟩ Plain text

```
thousands of files over time
→ COPY INTO can be suitable

millions+ files
→ Auto Loader preferred
```

γιατί Auto Loader κάνει file discovery πιο efficiently και επεξεργάζεται νέα files σε incremental batches. [🌐 Databricks Docu...](#)

Επίσης Auto Loader έχει πιο sophisticated schema inference/evolution. [🌐 Databricks Docu...](#)

## 32. Auto Loader — τι πρόβλημα λύνει;

Έστω folder:

⟨/⟩ Plain text

```
orders/
file000001
file000002
...
file5,000,000
```

Κάθε 10 sec προστίθενται:

⟨/⟩ Plain text

```
+100 files
```

Naïve listing:

⟨/⟩ Plain text

```
scan 5,000,000 names
to discover 100 new
```

κάθε φορά.

Δεν scaleάρει καλά.

Auto Loader κρατά **incremental state** για το τι έχει ήδη ανακαλύψει/επεξεργαστεί.

## 33. Auto Loader syntax

PySpark:

⟨/⟩ Python

```
orders = (
```

```
spark.readStream
 .format("cloudFiles")
 .option("cloudFiles.format", "json")
 .load("/Volumes/ecommerce/raw/orders/")
)
```

To magic identifier:

</> Plain text

```
cloudFiles
```

σημαίνει:

χρησιμοποίησε Auto Loader.

## 34. Γιατί `readStream`;

Auto Loader είναι built πάνω στο Spark Structured Streaming model.

Αυτό δεν σημαίνει απαραίτητα:

data source είναι Kafka/live event stream.

Τα source data είναι files.

Αλλά processing model είναι:

</> Plain text

```
incremental arrivals
↓
micro-batches
↓
new files
```

Άρα:

</> Python

```
spark.readStream
```

## 35. Full basic Auto Loader

</> Python

```
orders = (
 spark.readStream
 .format("cloudFiles")
 .option("cloudFiles.format", "json")
```

```
.option(
 "cloudFiles.schemaLocation",
 "/Volumes/ecommerce/checkpoints/orders_schema"
)
.load("/Volumes/ecommerce/raw/orders/")
)

query = (
 orders.writeStream
 .format("delta")
 .option(
 "checkpointLocation",
 "/Volumes/ecommerce/checkpoints/orders_bronze"
)
 .toTable("ecommerce.bronze.orders_raw")
)
```

Αυτό είναι πραγματικό production-style ingestion skeleton.

## 36. Τώρα δύο state locations

Εδώ οι beginners τα μπερδεύουν.

Έχουμε:

</> Plain text

```
cloudFiles.schemaLocation
```

και:

</> Plain text

```
checkpointLocation
```

Δεν είναι το ίδιο.

## 37. schemaLocation

Auto Loader πρέπει να θυμάται:

</> Plain text

```
ποιο schema ανακάλυψα;
πώς εξελίχθηκε;
ποια versions του schema είχα;
```

Άρα:

```
</> Python
```

```
.option(
 "cloudFiles.schemaLocation",
 "/Volumes/.../orders_schema"
)
```

εκεί κρατά schema metadata.

Η Databricks αποθηκεύει inferred schema σε `_schemas` κάτω από το configured `cloudFiles.schemaLocation`.

[Databricks Docu...](#)

## 38. Checkpoint

Checkpoint κρατά streaming progress/state.

Conceptually:

```
</> Plain text
```

```
files processed:
A
B
C
```

και:

```
</> Plain text
```

```
stream metadata/state
```

ώστε restart να συνεχίσει σωστά.

```
</> Python
```

```
.option(
 "checkpointLocation",
 "/Volumes/.../orders_checkpoint"
)
```

## 39. Schema location vs checkpoint

```
</> Plain text
```

```
schemaLocation
=
what shape does the incoming data have?
```

⟨/⟩ Plain text

```
checkpointLocation
=
how far has this streaming ingestion progressed?
```

Αυτή η διάκριση πρέπει να είναι απόλυτα καθαρή.

## 40. Restart example

Files:

⟨/⟩ Plain text

```
A
B
C
```

Stream processes:

⟨/⟩ Plain text

```
A
B
```

then cluster dies.

Checkpoint says:

⟨/⟩ Plain text

```
A, B done
```

Restart.

Auto Loader continues:

⟨/⟩ Plain text

```
C
```

Δεν ξεκινά από το μηδέν.

## 41. Why checkpoints must be persistent

Bad:

```
</> Plain text
```

```
/tmp/checkpoint
```

σε ephemeral local worker disk.

Cluster dies:

```
</> Plain text
```

```
checkpoint lost
```

Τώρα streaming state lost.

Άρα production checkpoints πρέπει να είναι σε persistent storage.

---

## 42. File discovery

Auto Loader πρέπει να μάθει:

ποια νέα files εμφανίστηκαν;

Υπάρχουν διαφορετικά file-discovery mechanisms.

Το βασικό conceptual split:

```
</> Plain text
```

```
directory listing
vs
event/file notification
```

---

## 43. Directory listing

Auto Loader εξετάζει storage directory listings για νέα files.

Conceptually:

```
</> Plain text
```

```
list path
↓
compare against tracked state
↓
discover new files
```

Απλό.

Αλλά σε τεράστια file counts μπορεί να είναι ακριβό.

## 44. File notification / managed file events

Cloud provider μπορεί να παράγει notifications όταν δημιουργείται file.

Conceptually:

⟨/⟩ Plain text

new file

↓

cloud event

↓

Auto Loader knows about it

Αυτό αποφεύγει repeated massive listing.

Η current Databricks architecture έχει managed file-event modes σε supported Unity Catalog external locations, αλλά επειδή exact cloud-specific implementation αλλάζει, το key conceptual takeaway είναι:

Auto Loader μπορεί να discover νέα files incrementally χωρίς full directory scan κάθε φορά.

## 45. Auto Loader file formats

Auto Loader supports schema inference/evolution για:

⟨/⟩ Plain text

JSON

CSV

XML

Avro

Parquet

με διαφορετικές semantics. [Databricks Docu...](#)

## 46. Το πολύ σημαντικό schema-inference detail

Για plain Spark DataFrameReader:

⟨/⟩ Python

```
spark.read
```

```
.option("inferSchema", "true")
```

σε CSV/JSON προσπαθεί να βρει actual types.

Auto Loader όμως by default για:

⟨/⟩ Plain text

```
JSON
CSV
XML
```

inferάρει fields ως:

```
</> Plain text

STRING
```

ώστε να μειώνει schema-evolution failures λόγω type mismatch. [Databricks Docu...](#)

Αυτό είναι exam-worthy και σημαντικό.

## 47. Example

JSON:

```
</> JSON

{"id": 1, "amount": 20.5}
```

Plain Spark inference μπορεί:

```
</> Plain text

id LONG
amount DOUBLE
```

Auto Loader default μπορεί να infer:

```
</> Plain text

id STRING
amount STRING
```

για schema-stability reasons. [Databricks Docu...](#)

## 48. Want actual column types?

Set:

```
</> Python

.option("cloudFiles.inferColumnTypes", "true")
```

Then Auto Loader uses type inference more like Spark's DataFrameReader for textual formats. [Databricks Docu...](#)

But in Bronze ingestion, keeping raw fields as strings is often intentionally safer.

## 49. Why Bronze often prefers strings

Suppose upstream sends:

</> Plain text

```
amount = 19.50
```

today.

Tomorrow:

</> Plain text

```
amount = "unknown"
```

If Bronze schema is:

</> Plain text

```
amount DOUBLE
```

parsing fails / rescues data.

If Bronze schema is:

</> Plain text

```
amount STRING
```

you preserve:

</> Plain text

```
"19.50"
```

```
"unknown"
```

and validate in Silver.

This fits medallion philosophy:

</> Plain text

```
Bronze:
```

```
preserve source
```

```
↓
```

```
Silver:
```

```
enforce clean types
```

## 50. Auto Loader initial schema inference

Auto Loader samples input.

By default it samples up to:

</> Plain text

```
50 GB
or
1000 files
```

whichever limit is reached first, for initial schema inference. [Databricks Docu...](#)

This is important:

it does not necessarily inspect the entire historical corpus just to infer schema.

## 51. Schema hints

Suppose Auto Loader infers:

</> Plain text

```
order_id STRING
amount STRING
```

but you want:

</> Plain text

```
order_id BIGINT
```

You can give hints.

</> Python

```
.option(
 "cloudFiles.schemaHints",
 "order_id BIGINT, amount DOUBLE"
)
```

This lets you influence inferred schema without necessarily declaring every column manually.

## 52. Important nuance about hints

For typed file formats like Parquet, schema hints can tell the Parquet reader which type to read a column as; they are not simply a post-read cast. [Databricks Docu...](#)

This matters because:

</> Plain text

```
schema hint
#
withColumn(cast(...))
```

semantically.

## 53. Schema evolution

Now source adds:

</> Plain text

```
discount_code
```

Yesterday:

</> Plain text

```
order_id
amount
```

Today:

</> Plain text

```
order_id
amount
discount_code
```

Auto Loader can detect and evolve schema depending on configured mode. [Databricks Docu...](#)

## 54. Why schema evolution can stop a stream

One common production behavior:

</> Plain text

```
new unexpected column discovered
↓
Auto Loader records schema change
↓
stream fails/restarts depending configuration
```

↓

new schema used after restart

This is actually desirable in many production contexts:

don't silently mutate pipeline semantics without control.

## 55. Rescued data

This is one of the best ingestion concepts in Databricks.

Suppose expected schema:

</> Plain text

```
id BIGINT
amount DOUBLE
```

Incoming:

</> JSON

```
{
 "id": 10,
 "amount": "UNKNOWN",
 "new_field": "abc"
}
```

Instead of silently throwing away malformed/unexpected fields, Databricks can place them into a **rescued data column**.

The current `read_files` /Auto Loader behavior supports a rescued data column specifically for values or fields that don't match the expected schema. [🌐 Databricks Docu... +1](#)

## 56. Conceptual rescued row

</> Plain text

```
id = 10
amount = null

_rescued_data =
{
 "amount": "UNKNOWN",
 "new_field": "abc"
}
```

Now you haven't lost source information.

## 57. Why rescued data is valuable

Without rescue:

</> Plain text

```
bad row
↓
drop?
fail?
corrupt?
```

With rescue:

</> Plain text

```
valid columns
+
unexpected content preserved
```

Then Silver pipeline can:

</> Plain text

```
inspect
alert
quarantine
fix
```

---

## 58. Bronze architecture pattern

A robust Bronze table might contain:

</> Plain text

```
business/source fields
+
ingestion timestamp
+
source filename
+
rescued data
```

Example:

</> Plain text

```
order_id
customer_id
amount
status
_ingested_at
```

```
_source_file
_rescued_data
```

This gives excellent auditability.

## 59. Quarantine pattern

Rows with:

```
</> Plain text

_rescued_data IS NOT NULL
```

may be routed into:

```
</> Plain text

quarantine.orders_bad
```

while clean rows continue to Silver.

Conceptually:

```
</> Plain text

Bronze
 ↓
validation
├─ clean → Silver
└─ bad → Quarantine
```

## 60. Explicit schema with Auto Loader

You don't have to infer.

```
</> Python

schema = """
order_id BIGINT,
customer_id BIGINT,
amount STRING,
status STRING,
order_ts STRING
"""

orders = (
 spark.readStream
 .format("cloudFiles")
 .option("cloudFiles.format", "json")
 .schema(schema)
 .load(source)
```

```
)
```

This is very common in production where upstream contract is known.

## 61. schemaLocation still useful?

If you're fully providing schema and not using Auto Loader schema inference/evolution, `schemaLocation` may not be required in the same way.

But whenever using Auto Loader-managed inference/evolution, `cloudFiles.schemaLocation` is critical.

## 62. Exactly-once semantics — be careful

Auto Loader is designed for exactly-once file ingestion semantics into Delta when combined correctly with checkpointing and supported sink behavior.

But this does **not** mean:

    duplicates can never appear in your final business table.

Why?

Because duplicates can originate in source data itself.

Example:

```
</> Plain text
```

```
file A:
order_id=100
```

```
file B:
order_id=100
```

Both are distinct files and both legitimately processed once.

Result:

```
</> Plain text
```

```
two business rows with order_id=100
```

So:

```
</> Plain text
```

```
file-level exactly once
≠
business-key deduplication
```

## 63. Very important distinction

Auto Loader tracks:

```
</> Plain text
```

```
files
```

not:

```
</> Plain text
```

```
business entities
```

So:

```
</> Plain text
```

```
fileA processed once
fileB processed once
```

doesn't imply:

```
</> Plain text
```

```
order_id 100 appears once
```

Silver must often deduplicate/upsert.

## 64. Auto Loader + Bronze + MERGE

Common architecture:

```
</> Plain text
```

```
Files
↓
Auto Loader
↓
Bronze append
↓
dedup / CDC
↓
MERGE
↓
Silver
```

This cleanly separates:

Plain text

```
ingestion correctness
```

from:

Plain text

```
business-state correctness
```

## 65. AvailableNow pattern

Sometimes you don't want a permanently running stream.

You want:

Plain text

```
process all new files now
then stop
```

Structured Streaming/Auto Loader supports triggered incremental processing patterns such as AvailableNow in supported environments.

Conceptually:

Plain text

```
start
↓
process backlog
↓
reach latest known input
↓
stop
```

Useful for scheduled batch-like incremental ingestion.

This is a beautiful example that:

streaming engine can implement batch-like incremental jobs.

## 66. COPY INTO vs Auto Loader mental model

COPY INTO :

Plain text

```
run command
```

```
↓
discover unprocessed source files
↓
load
↓
finish
```

Auto Loader:

</> Plain text

```
stateful incremental stream
↓
discover files continuously/in batches
↓
checkpoint progress
↓
load incrementally
```

## 67. When to use COPY INTO

Use when:

</> Plain text

```
moderate number of files
SQL-centric workflow
simple ingestion
you want easy retrievable/idempotent loading
reloading selected files matters
```

Databricks explicitly notes `COPY INTO` can work well for thousands of files over time. [🌐 Databricks Docu...](#)

## 68. When to use Auto Loader

Use when:

</> Plain text

```
large / continuously growing file source
millions of files
frequent arrivals
schema changes
streaming/incremental pipelines
high-scale production ingestion
```

Databricks recommends Auto Loader for file counts in the millions or more over time and for more sophisticated schema evolution. [🌐 Databricks Docu...](#)

## 69. A common misconception

Auto Loader = faster `spark.read` .

Not exactly.

Its main advantage is:

</> Plain text

```
incremental file discovery
+
state tracking
+
schema handling
+
streaming integration
```

rather than simply reading each individual file faster.

---

## 70. Another misconception

Auto Loader turns files into Kafka.

No.

Source remains files.

The processing model is streaming/incremental.

---

## 71. Production example — Bronze ingestion

</> Python

```
from pyspark.sql import functions as F

source_path = "/Volumes/ecommerce/raw/orders/"
schema_path = "/Volumes/ecommerce/system/orders_schema/"
checkpoint_path = "/Volumes/ecommerce/system/orders_checkpoint/"

orders = (
 spark.readStream
 .format("cloudFiles")
 .option("cloudFiles.format", "json")
 .option("cloudFiles.schemaLocation", schema_path)
 .load(source_path)
 .withColumn("_ingested_at", F.current_timestamp())
)

query = (
 orders.writeStream
 .format("delta")
 .option("checkpointLocation", checkpoint_path)
```

```
.toTable("ecommerce.bronze.orders_raw")
)
```

## 72. What happens internally?

Initial run:

</> Plain text

1. discover files
2. infer schema
3. persist schema state
4. decide which files are new
5. process batch
6. write Delta transaction
7. update checkpoint

Next run:

</> Plain text

1. read checkpoint
2. discover files
3. skip already-processed files
4. process only new
5. commit
6. advance checkpoint

That is the heart of Auto Loader.

## 73. What if checkpoint is deleted?

This is a classic production incident.

Suppose:

</> Plain text

- A
- B
- C

already processed.

You accidentally delete checkpoint.

Next start has no ingestion progress history.

Potential consequence:

</> Plain text

```
files may be seen as new
```

and depending on pipeline/sink behavior you may duplicate ingestion or need recovery/reconciliation.

Therefore:

checkpoint is operational state, not disposable temp data.

---

## 74. Checkpoint path reuse

Another bad practice:

</> Plain text

```
Pipeline A
checkpoint=/shared/checkpoint
```

```
Pipeline B
checkpoint=/shared/checkpoint
```

Two unrelated queries sharing checkpoint path can corrupt semantics/state.

Rule:

each streaming query gets its own checkpoint location.

---

## 75. Changing source with same checkpoint

Suppose checkpoint belongs to:

</> Plain text

```
/orders/sourceA
```

then you change code to:

</> Plain text

```
/orders/sourceB
```

while keeping same checkpoint.

Dangerous.

Checkpoint state belongs to the logical streaming query/source topology.

Don't treat it like generic cache.

## 76. Debugging case: new files exist but no rows appear

Start systematically.

⌕ Plain text

Source

↓

Do new files actually exist?

↓

Does path match?

↓

Are extensions/formats correct?

↓

Did Auto Loader discover them?

↓

Checkpoint says already processed?

↓

Parsing failure?

↓

Schema issue?

↓

Sink write failure?

## 77. Debugging case: file accidentally overwritten

COPY INTO may still consider a previously loaded source file as already processed even if its contents changed.

🌐 Databricks Docu...

That means source systems should ideally follow:

⌕ Plain text

immutable file delivery

rather than:

⌕ Plain text

upload file.csv

later mutate file.csv

later mutate again

Immutable landing zones are much easier to reason about.

## 78. Best practice: source files immutable

Prefer:

</> Plain text

```
orders_20260928_001.json
orders_20260928_002.json
```

over:

</> Plain text

```
orders_latest.json
```

being overwritten repeatedly.

Why?

Because incremental ingestion systems reason much more reliably about:

</> Plain text

```
new file identity
```

than mutable files.

---

## 79. Debugging case: schema suddenly changes

Yesterday:

</> Plain text

```
amount STRING
```

Today source sends:

</> Plain text

```
amount STRUCT
```

Questions:

</> Plain text

```
was source contract intentionally changed?
did only one corrupt file do this?
should field be rescued?
should schema evolve?
should stream fail?
```

Do not blindly allow everything.

---

## 80. Debugging case: new column missing

Upstream adds:

</> Plain text

```
discount
```

but Bronze doesn't show it.

Check:

</> Plain text

```
schema evolution mode
schemaLocation state
provided explicit schema
rescued data
```

If you supplied strict explicit schema, new field may not appear as regular column automatically.

---

## 81. Debugging case: duplicated business IDs

Rows:

</> Plain text

```
order_id=100
order_id=100
```

Do not immediately blame Auto Loader.

Check:

</> Plain text

```
same source file loaded twice?
two different files both contain ID?
upstream retry?
Silver merge/dedup missing?
checkpoint reset?
```

This is exactly why we separate:

</> Plain text

```
file idempotency
vs
```

```
business-key idempotency
```

## 82. Debugging case: malformed CSV

Expected:

```
</> csv

id,amount
1,10.5
```

Bad:

```
</> csv

2,"10.5
```

Possible result:

```
</> Plain text

parse error
corrupt/rescued data
unexpected columns
```

Investigate parser options:

```
</> Plain text

quote
escape
multiline
delimiter
header
```

## 83. Source file metadata as audit evidence

Good Bronze systems often add:

```
</> Plain text

_source_file
_ingestion_ts
```

so when Silver row is wrong you can trace:

```
</> Plain text
```

```
Silver bad row
↓
Bronze source row
↓
original file
```

This is exactly the kind of operational observability production pipelines need.

## 84. Ingestion ≠ transformation

Bronze ingestion should often be conservative.

Bad Bronze:

</> Plain text

```
ingest
+
business rules
+
join 8 tables
+
calculate revenue
+
drop malformed source data
```

Better:

</> Plain text

```
Bronze:
land faithfully

Silver:
validate, dedup, cast, join, apply CDC
```

That separation makes recovery easier.

## 85. read\_files + streaming tables preview

Later in Lakeflow you will write things like:

</> SQL

```
CREATE OR REFRESH STREAMING TABLE bronze_orders
AS
SELECT *
FROM STREAM read_files(
 '/Volumes/ecommerce/raw/orders/',
 format => 'json'
);
```

When `read_files` is used in streaming-table context, Databricks uses Auto Loader for incremental file ingestion.

⊕ Databricks Docu...

So what you're learning now directly becomes Phase 6 syntax later.

## 86. Production architecture

A good ingestion architecture:

⌵ Plain text

```
Cloud / Volume Landing Zone
 ↓
immutable files
 ↓
Auto Loader
 ↓
Bronze
raw + metadata + rescued data
 ↓
validation
 ↓
Silver
typed + dedup + CDC
 ↓
Gold
```

This is the architecture I want you to internalize.

## 87. Practical lab — CSV ingestion

Suppose file:

⌵ csv

```
order_id,customer_id,amount,status
1001,10,19.50,NEW
1002,11,25.00,PAID
```

Explicit schema:

⌵ Python

```
schema = """
order_id BIGINT,
customer_id BIGINT,
amount DOUBLE,
status STRING
"""
```

Read:

**</> Python**

```
df = (
 spark.read
 .format("csv")
 .option("header", "true")
 .schema(schema)
 .load("/Volumes/ecommerce/raw/orders_csv/")
)

display(df)
```

## 88. Practical lab — plain batch load

**</> Python**

```
(
 df.write
 .mode("append")
 .saveAsTable("ecommerce.bronze.orders_raw")
)
```

Run twice.

What happens?

If same source file is re-read manually:

**</> Plain text**

```
duplicate rows
```

because plain `spark.read` has no inherent file-history memory.

This is intentional experiment.

## 89. Practical lab — COPY INTO

Now clear/rebuild table and run:

**</> SQL**

```
COPY INTO ecommerce.bronze.orders_raw
FROM '/Volumes/ecommerce/raw/orders_csv/'
FILEFORMAT = CSV
FORMAT_OPTIONS (
 'header' = 'true',
 'inferSchema' = 'true'
);
```

Run twice.

Second time, previously loaded files should be skipped under normal `COPY INTO semantics`. [Databricks Docu...](#) +1

## 90. Practical lab — validate first

`</>` SQL

```
COPY INTO ecommerce.bronze.orders_raw
FROM '/Volumes/ecommerce/raw/orders_csv/'
FILEFORMAT = CSV
FORMAT_OPTIONS (
 'header' = 'true'
)
VALIDATE ALL;
```

Check the result before actual load.

## 91. Practical lab — Auto Loader

`</>` Python

```
source = "/Volumes/ecommerce/raw/orders_json/"
schema_loc = "/Volumes/ecommerce/system/orders_schema/"
checkpoint = "/Volumes/ecommerce/system/orders_checkpoint/"

orders = (
 spark.readStream
 .format("cloudFiles")
 .option("cloudFiles.format", "json")
 .option("cloudFiles.schemaLocation", schema_loc)
 .load(source)
)

query = (
 orders.writeStream
 .format("delta")
 .option("checkpointLocation", checkpoint)
 .toTable("ecommerce.bronze.orders_raw")
)
```

Add new file.

Observe that only new arrivals are processed.

## 92. Practical lab — schema evolution experiment

Initial file:

```
</> JSON
```

```
{"id":1,"amount":"20.0"}
```

Later file:

```
</> JSON
```

```
{"id":2,"amount":"25.0","discount":"SAVE10"}
```

Observe:

```
</> Plain text
```

```
schemaLocation
new schema
possible stream behavior
```

and inspect rescued data/schema evolution behavior.

---

## 93. Practical lab — bad data

Add:

```
</> JSON
```

```
{"id":3,"amount":{"unexpected":true}}
```

Inspect:

```
</> Plain text
```

```
_rescued_data
```

or failure behavior depending on your configured schema/mode.

The goal:

don't just make the pipeline pass; understand where unexpected data goes.

---

## 94. COPY INTO vs Auto Loader comparison

Property	spark.read	COPY INTO	Auto Loader
Basic batch read	✓	✓	via streaming model
Remembers processed files	✗	✓	✓
Idempotent file load	✗ by itself	✓	✓ with checkpoint/sink semantics
SQL-friendly	limited	✓	SQL via Lakeflow/read_files
Huge file counts	weak/manual	okay moderate scale	strongest
Continuous ingestion	✗	repeated runs	✓
Schema evolution	manual	supported	strongest/flexible
Checkpoint	✗	internal tracking	explicit streaming checkpoint
Streaming integration	✗	✗ as stream	✓

## 95. Common exam trap

Question:

You have millions of files arriving continuously in cloud object storage. Which ingestion mechanism should you choose?

Best answer:

</> Plain text

Auto Loader

not repeated `spark.read` , and typically not `COPY INTO` at that scale. [Databricks Docu...](#)

## 96. Another exam trap

Question:

You rerun `COPY INTO` against the same source directory. What happens to already-loaded files?

They are skipped by default. [Databricks Docu...](#) +1

## 97. Another exam trap

`cloudFiles.schemaLocation` stores:

Plain text

```
schema history/state
```

not:

Plain text

```
stream progress
```

The latter belongs to checkpoint.

---

## 98. Another exam trap

Auto Loader:

Plain text

```
CSV/JSON default inference
```

typically uses strings to reduce type mismatch/evolution problems unless `cloudFiles.inferColumnTypes=true` .

[Databricks Docu...](#)

---

## 99. Engineering question

Why would you prefer explicit schema in production?

Strong answer:

It avoids an inference pass, gives a stable data contract, makes failures predictable, prevents accidental type drift, and allows schema changes to be reviewed instead of silently inferred.

This aligns with Databricks' documentation that supplying a schema lets the reader skip inference. [Databricks Docu...](#)

---

## 100. Engineering question

Why not write Auto Loader directly into Silver?

Because Bronze is useful for:

Plain text

```
source fidelity
replay
audit
schema drift
rescued data
debugging
```

Silver can then enforce:

</> Plain text

```
business keys
types
deduplication
CDC
quality
```

This separation minimizes coupling between ingestion and business transformation.

## 101. Engineering question

What exactly does checkpoint protect?

Strong answer:

It stores the streaming query's progress and state so that after restart the query can continue from the correct point rather than rediscovering the stream as if it were new.

## 102. Engineering question

Why doesn't file-level exactly-once guarantee no duplicate business rows?

Because:

</> Plain text

```
file identity
#
business entity identity
```

Two distinct source files can legitimately contain the same `order_id`.

Therefore deduplication belongs downstream.

## 103. Big test — Phase 4

### Q1

What's the main difference between `spark.read` and Auto Loader?

**Answer:** `spark.read` is ordinary batch file reading; Auto Loader adds incremental discovery/stateful file processing on Structured Streaming.

### Q2

Why explicit schema?

**Answer:** Performance, predictable contract, easier debugging, controlled evolution. [🌐 Databricks Docu...](#)

### Q3

What happens if you use `spark.read` over the same whole directory every day?

**Answer:** It reads the current directory contents again unless you manually restrict the input.

### Q4

What does `COPY INTO` remember?

**Answer:** Which source files it has already loaded, allowing retryable/idempotent ingestion. [🌐 Databricks Docu...](#)

### Q5

Does `COPY INTO` reload a previously loaded file just because its contents changed?

**Answer:** No, not by default. Already-loaded files are skipped even if modified. [🌐 Databricks Docu...](#)

### Q6

What's `PATTERN` ?

**Answer:** A filter for source filenames.

### Q7

What's `VALIDATE` ?

**Answer:** Test parse/schema/constraints without loading data. [🌐 Databricks Docu...](#)

### Q8

When is Auto Loader better than `COPY INTO` ?

**Answer:** Continuous/high-scale ingestion, millions of files, richer schema evolution. [🌐 Databricks Docu...](#)

### Q9

What does `cloudFiles` mean?

**Answer:** Auto Loader source provider.

### Q10

Why `readStream` for file ingestion?

**Answer:** Because Auto Loader uses the Structured Streaming incremental-processing model.

### Q11

What is `schemaLocation` ?

**Answer:** Persistent schema inference/evolution metadata.

### Q12

What is `checkpointLocation` ?

**Answer:** Streaming progress/state location.

### Q13

Are they the same?

**Answer:** No.

### Q14

What happens if checkpoint disappears?

**Answer:** The stream loses its prior progress/state and may require recovery/reconciliation; duplicate ingestion can become possible depending on restart strategy.

### Q15

Why must checkpoints be persistent?

**Answer:** They must survive compute restart/termination.

### Q16

Why not share one checkpoint between different streams?

**Answer:** Checkpoint state belongs to one logical streaming query.

### Q17

How does Auto Loader infer CSV/JSON types by default?

**Answer:** Typically as strings unless `cloudFiles.inferColumnTypes=true` . [⊕ Databricks Docu...](#)

### Q18

How much input does Auto Loader sample by default for initial schema inference?

**Answer:** Up to 50 GB or 1000 files, whichever threshold is reached first. [⊕ Databricks Docu...](#)

### Q19

What is rescued data?

**Answer:** Unexpected/mismatched data preserved in a special column instead of being silently lost. [⊕ Databricks Docu...](#)

### Q20

Why useful?

**Answer:** Auditability, schema-drift handling, debugging, quarantine.

### Q21

Does exactly-once file ingestion mean no duplicate `customer_id` ?

**Answer:** No.

### Q22

Where should business-key dedup generally happen?

**Answer:** Silver / CDC processing layer.

## Q23

What is the best architecture for file ingestion?

**Answer:**

Plain text

```
Landing zone
→ Auto Loader
→ Bronze
→ validate/dedup/CDC
→ Silver
→ Gold
```

## Q24

Why use immutable files?

**Answer:** Incremental systems reason much more reliably about new immutable file identities than repeatedly mutated filenames.

## Q25

What is the difference between `FORMAT_OPTIONS mergeSchema` and `COPY_OPTIONS mergeSchema` ?

**Answer:** The former helps reconcile source-file schemas; the latter evolves target Delta schema. [Databricks Docu...](#) +1

## Production assignment

Build this ingestion flow:

Plain text

```
/Volumes/ecommerce/raw/orders/
↓
Auto Loader
↓
ecommerce.bronze.orders_raw
```

Bronze should include:

Plain text

```
order_id
customer_id
amount
status
_ingested_at
_rescued_data
```

Then deliberately test:

**</> Plain text**

1. ingest first JSON file
2. restart stream
3. add second file
4. verify first wasn't reprocessed
5. add a new column
6. observe schema evolution
7. add malformed value
8. inspect rescued data
9. delete checkpoint in a sandbox only
10. observe changed behavior
11. restore/recreate safely
12. add duplicate business key in a different file
13. verify Auto Loader processes each file once
14. prove business duplicate still exists
15. deduplicate downstream

That covers the Phase 4 roadmap exactly: explicit schemas vs inference, file reads, `COPY INTO`, Auto Loader, discovery, checkpoints, `schemaLocation`, schema hints/evolution, rescued data, duplicate prevention and incremental ingestion semantics. [databricks\\_mastery\\_phased\\_plan](#)

The next roadmap phase is **Phase 5 — Structured Streaming**, where we'll go beneath Auto Loader and understand `readStream` / `writeStream`, micro-batches, offsets, state, watermarks, event time vs processing time, late data, append/update/complete modes, exactly-once semantics and state-store debugging. [databricks\\_mastery\\_phased\\_plan](#)

[Continue to next phase](#)

Worked for 12s

## Phase 5 — Structured Streaming

Προχωράμε ακριβώς σύμφωνα με το roadmap σου. Η **Phase 5** έχει στόχο να καταλάβεις streaming από το execution model μέχρι production reliability: `readStream`, `writeStream`, micro-batches, checkpoints, offsets, state, watermarks, event time, late data, output modes, stateless/stateful transformations, exactly-once semantics, restart behavior και state-store debugging.

[databricks\\_mastery\\_phased\\_plan](#)

Το βασικό mental model αυτής της φάσης είναι:

**</> Plain text**

```
STREAMING SOURCE
 ↓
readStream
 ↓
incremental DataFrame plan
 ↓
trigger
 ↓
micro-batch / real-time execution
 ↓
stateless or stateful operators
 ↓
writeStream
 ↓
sink
```



checkpoint  
remembers progress + state

Το πιο σημαντικό: **Structured Streaming δεν είναι διαφορετικό Spark**. Είναι το ίδιο Spark execution engine, αλλά αντί να επεξεργάζεται ένα bounded dataset μία φορά, επεξεργάζεται ένα dataset που λογικά συνεχίζει να μεγαλώνει. Η Databricks διαχωρίζει ρητά stateless και stateful streaming, checkpoints, output modes, trigger intervals και watermarks ως τα βασικά building blocks. [🌐 Databricks Docu...](#)

## 1. Batch vs streaming από first principles

Batch:

⌕ Plain text

input known now:

A  
B  
C  
D

↓ process

result

↓ stop

To dataset είναι **bounded**.

Streaming:

⌕ Plain text

10:00 A B  
10:01 C  
10:02 D E  
10:03 F  
...

Δεν ξέρουμε πού τελειώνει.

To dataset είναι λογικά:

⌕ Plain text

unbounded

Structured Streaming σου επιτρέπει να γράψεις transformations περίπου όπως batch DataFrames:

⌕ Python

```
result = (
 stream_df
 .filter("amount > 0")
 .groupBy("country")
 .count()
)
```

αλλά Spark κρατά execution progress και, όπου χρειάζεται, intermediate state μεταξύ των διαφορετικών arrivals.

## 2. Το πρώτο streaming DataFrame

Ένα batch DataFrame:

```
</> Python

df = (
 spark.read
 .format("json")
 .load(path)
)
```

Streaming:

```
</> Python

df = (
 spark.readStream
 .format("json")
 .schema(schema)
 .load(path)
)
```

Η μικρή διαφορά:

```
</> Plain text

read
vs
readStream
```

αλλάζει εντελώς το execution model.

Με read :

«διάβασε ό,τι υπάρχει τώρα».

Με readStream :

«θεώρησε αυτό το source μία συνεχώς εξελισσόμενη πηγή και κράτα track της προόδου».

### 3. readStream δεν ξεκινά από μόνο του execution

Αυτό είναι Spark lazy evaluation ξανά.

</> Python

```
events = spark.readStream.format(...).load(...)
```

δημιουργεί streaming DataFrame / logical plan.

Το πραγματικό streaming execution ξεκινά όταν κάνεις:

</> Python

```
query = (
 events.writeStream
 .format("delta")
 .option("checkpointLocation", checkpoint)
 .toTable("ecommerce.bronze.events")
)
```

Άρα:

</> Plain text

```
readStream
→ define source

transformations
→ define computation

writeStream
→ define sink + start streaming query
```

### 4. Τι είναι micro-batch;

Αυτό είναι ίσως το σημαντικότερο concept.

Έστω events φτάνουν:

</> Plain text

```
12:00:00 e1
12:00:01 e2
12:00:02 e3
12:00:03 e4
12:00:04 e5
```

Spark δεν χρειάζεται να ξεκινήσει νέο distributed job για κάθε ένα event.

Μπορεί να τα ομαδοποιήσει:

</> Plain text

```
Micro-batch 1:
e1 e2
```

```
Micro-batch 2:
e3 e4
```

```
Micro-batch 3:
e5
```

Κάθε micro-batch εκτελείται σαν μικρό Spark job.

Conceptually:

</> Plain text

```
new input
 ↓
Micro-batch N
 ↓
Spark Job
 ↓
Stages
 ↓
Tasks
 ↓
Partitions
```

Άρα όσα έμαθες:

</> Plain text

```
job
stage
task
partition
shuffle
join
spill
skew
```

εξακολουθούν να ισχύουν.

## 5. Παράδειγμα: 5-second micro-batches

Σε classic compute:

</> Python

```
query = (
 events.writeStream
 .format("delta")
```

```
.outputMode("append")
.option("checkpointLocation", checkpoint)
.trigger(processingTime="5 seconds")
.toTable("ecommerce.bronze.events")
)
```

Conceptually:

</> Plain text

00–05 sec

↓

batch 0

05–10 sec

↓

batch 1

10–15 sec

↓

batch 2

Η `processingTime` trigger λέει πόσο συχνά να γίνεται προσπάθεια για νέο micro-batch. [Databricks Docu...](#)

## 6. Αν ένα batch χρειάζεται περισσότερο από το trigger interval;

Έχεις:

</> Plain text

```
trigger = every 5 seconds
```

αλλά ένα micro-batch χρειάζεται:

</> Plain text

```
12 seconds
```

Δεν τρέχει magically τρία batches ταυτόχρονα στο normal sequential micro-batch model.

Γίνεται περίπου:

</> Plain text

Batch 1:

0 → 12 sec

Batch 2:

starts after Batch 1

Άρα αρχίζει να δημιουργείται:

</> Plain text

processing backlog

και streaming latency αυξάνεται.

Αυτό είναι production signal:

ingestion rate > processing capacity.

## 7. Trigger types

Σήμερα τα σημαντικότερα trigger models είναι:

Trigger	Τι σημαίνει
Default / processingTime=0	ξεκίνα επόμενο micro-batch όσο πιο γρήγορα γίνεται
processingTime="10 seconds"	fixed-interval micro-batches
availableNow=True	επεξεργάσου ό,τι είναι διαθέσιμο τώρα και σταμάτα
real-time mode	ultra-low-latency operational streaming σε supported classic environments

Η Databricks έχει deprecated το `Trigger.Once` υπέρ του `Trigger.AvailableNow`. Επίσης, σε **serverless notebooks/jobs** υποστηρίζονται `AvailableNow` και το deprecated `Once`, όχι time-based `ProcessingTime`; για continuously running serverless workloads η Databricks κατευθύνει προς Lakeflow pipelines continuous mode. [Databricks Docu...](#) +1

## 8. AvailableNow — πολύ σημαντικό για Data Engineering

Έστω κάθε ώρα θέλεις:

επεξεργάσου όλα τα νέα records από την τελευταία φορά και μετά σταμάτα.

</> Python

query = (  
 events.writeStream  
 .trigger(availableNow=True)  
 .option("checkpointLocation", checkpoint)  
 .toTable("ecommerce.bronze.events")  
)

Run 1:

</> Plain text

available:

```
A B C

process A B C
stop
```

Μία ώρα μετά:

```
</> Plain text

available:
D E

checkpoint knows A B C

process D E
stop
```

Αυτό είναι εξαιρετικά σημαντικό γιατί δείχνει:

**Structured Streaming δεν σημαίνει απαραίτητα always-on process.**

Μπορείς να χρησιμοποιείς το streaming engine για **scheduled incremental batch processing**. Η Databricks το προτείνει ιδιαίτερα για serverless jobs. [🌐 Databricks Docu... +1](#)

## 9. Γιατί να χρησιμοποιήσω streaming engine για scheduled batch;

Γιατί σου δίνει:

```
</> Plain text

incremental source tracking
checkpointing
state management
fault recovery
incremental semantics
```

χωρίς εσύ να γράψεις:

```
</> Python

last_processed_timestamp = ...
WHERE event_ts > ...
```

και να διαχειρίζεσαι manually progress state.

Αυτό είναι ακριβώς το μοντέλο που είδαμε στο Auto Loader.

## 10. Stateless streaming

Παράδειγμα:

</> Python

```
clean = (
 events
 .filter("amount > 0")
 .select("event_id", "customer_id", "amount")
)
```

Για κάθε record μπορείς να αποφασίσεις τι θα κάνεις χωρίς να θυμάσαι previous records.

</> Plain text

```
event
↓
filter
↓
projection
↓
output
```

Αυτό είναι **stateless**.

Η Databricks ορίζει stateless queries ως queries που δεν κρατούν intermediate state πέρα από το progress tracking από source σε sink. [Databricks Docu...](#)

## 11. Stateful streaming

Τώρα:

</> Python

```
events.groupBy("customer_id").count()
```

Για να υπολογίσει:

</> Plain text

```
customer 42 count = ?
```

πρέπει να θυμάται:

</> Plain text

```
Batch 1:
customer 42 = 5
```

```
Batch 2:
+3
```

```
new total = 8
```

Άρα Spark κρατά **state**.

Stateful operators περιλαμβάνουν aggregations, distinct , dropDuplicates , stream-stream joins και custom stateful operations. [Databricks Docu...](#)

## 12. Τι είναι state πραγματικά;

State = information από προηγούμενα micro-batches που χρειάζεται για να επεξεργαστεί σωστά τα επόμενα.

Παράδειγμα:

</> Plain text

key	count
customer_1	50
customer_2	22
customer_3	910
...	

Αυτό δεν είναι απλώς final output.

Είναι working memory του streaming computation.

## 13. Γιατί το state μπορεί να γίνει τεράστιο;

Έστω:

</> Python

```
events.groupBy("customer_id").count()
```

και κάθε μέρα εμφανίζονται 1 εκατομμύριο νέα customers.

Αν δεν υπάρχει λόγος να ξεχάσεις ποτέ παλιούς customers:

</> Plain text

Day 1	1M keys
Day 30	30M keys
Day 365	365M keys

State μεγαλώνει συνεχώς.

Αποτέλεσμα:

</> Plain text

```
more memory
larger checkpoints
```

```
slower state lookup
slower batches
potential OOM
```

Αυτό είναι γιατί τα **watermarks** είναι τόσο σημαντικά.

## 14. Processing time vs event time

Αυτό πρέπει να είναι απόλυτα καθαρό.

Έστω event:

```
</> JSON

{
 "event_id": 123,
 "event_time": "2026-09-28 10:03:00"
}
```

φτάνει στο Databricks στις:

```
</> Plain text

10:11:00
```

Έχουμε δύο διαφορετικούς χρόνους.

### Event time

```
</> Plain text

10:03
```

Πότε συνέβη το business event.

### Processing time

```
</> Plain text

10:11
```

Πότε το processing system το είδε/επεξεργάστηκε.

## 15. Γιατί μπορεί να διαφέρουν;

Π.χ.:

```
</> Plain text
```

```
mobile offline
network delay
Kafka backlog
server outage
retry
batch upload
clock skew
```

Ένας χρήστης μπορεί να αγοράσει στις:

</> Plain text

10:03

αλλά το event να φτάσει στις:

</> Plain text

10:11

Άρα το record είναι **late-arriving**.

## 16. Event-time windows

Θέλεις sales per 10-minute window:

</> Python

```
from pyspark.sql import functions as F

sales = (
 events
 .groupBy(
 F.window("event_time", "10 minutes")
)
 .agg(
 F.sum("amount").alias("revenue")
)
)
```

Windows:

</> Plain text

10:00–10:10  
10:10–10:20  
10:20–10:30

Το event που έγινε:

```
</> Plain text
```

```
10:03
```

ανήκει:

```
</> Plain text
```

```
10:00-10:10
```

ακόμα κι αν έφτασε στις 10:11.

---

## 17. To late-data problem

Suppose window:

```
</> Plain text
```

```
10:00-10:10
```

Στις:

```
</> Plain text
```

```
10:10
```

μπορείς να πεις:

η window τελείωσε.

Όμως τι γίνεται αν στις:

```
</> Plain text
```

```
10:15
```

έρθει event με:

```
</> Plain text
```

```
event_time = 10:08
```

;

Ανήκει στο παλιό window.

Άρα έχεις tradeoff:

</> Plain text

```
wait forever
→ correct for arbitrarily late data
→ infinite state

close immediately
→ low state
→ lose late data
```

Χρειαζόμαστε compromise.

## 18. Watermark

</> Python

```
watermarked = (
 events
 .withWatermark(
 "event_time",
 "10 minutes"
)
)
```

Meaning approximately:

Είμαι διατεθειμένος να κρατώ state ώστε να δεχθώ events που καθυστερούν περίπου μέχρι 10 λεπτά σε σχέση με το observed event-time progress.

Η Databricks ορίζει watermark ως event-time threshold που επιτρέπει στο Structured Streaming να σταματά να κρατά παλιό state και να ελέγχει late-arriving data. [Databricks Docu...](#) +1

## 19. Πολύ σημαντικό: watermark ≠ wall-clock timer

Suppose:

</> Python

```
.withWatermark("event_time", "10 minutes")
```

Δεν σημαίνει απλώς:

κάθε event έχει 10 πραγματικά λεπτά από τη στιγμή που γεννήθηκε.

Spark παρακολουθεί το **maximum event time που έχει δει** και υπολογίζει το watermark σε σχέση με αυτό. [Databricks Docu...](#)

Αν maximum seen event time είναι:

</> Plain text

12:00

με delay:

</> Plain text

10 min

conceptual watermark:

</> Plain text

≈ 11:50

Άρα sufficiently old state μπορεί να θεωρηθεί final/evictable.

## 20. Γιατί δεν λέμε «ακριβώς 10 λεπτά»;

Η current API documentation λέει ότι το actual watermark είναι guaranteed να βρίσκεται τουλάχιστον όσο το configured delay πίσω από το actual event-time progress· coordination μεταξύ partitions σημαίνει ότι το exact instantaneous value δεν πρέπει να αντιμετωπίζεται σαν τέλειο wall-clock cutoff. [Databricks Docu...](#)

Για interview:

Watermark is an event-time progress threshold, not a simple timer per row.

## 21. Stateful aggregation με watermark

</> Python

```
from pyspark.sql import functions as F

sales = (
 events
 .withWatermark("event_time", "15 minutes")
 .groupBy(
 F.window("event_time", "10 minutes"),
 F.col("country")
)
 .agg(
 F.sum("amount").alias("revenue")
)
)
```

Spark state might conceptually contain:

</> Plain text

window                      country                      revenue

```

10:00-10:10 GR 1200
10:00-10:10 DE 800
10:10-10:20 GR 400
...
```

Όταν watermark περάσει sufficiently beyond old windows:

</> Plain text

```
old window state
→ can be removed
```

Έτσι state μένει bounded.

## 22. Short vs long watermark

Suppose:

</> Plain text

```
5-minute watermark
```

Advantages:

</> Plain text

```
less state
faster state operations
lower memory
lower latency in append finalization
```

Disadvantage:

</> Plain text

```
late records > tolerance may be dropped
```

Long:

</> Plain text

```
2 hours
```

Advantages:

</> Plain text

```
accept more late events
```

Costs:

</> Plain text

```
more state
more memory
more checkpoint size
higher latency
```

Η Databricks περιγράφει ακριβώς αυτό το latency/state/correctness tradeoff και προειδοποιεί ότι πολύ μικρό watermark μπορεί να απορρίψει valid late records. [Databricks Docu...](#) +1

## 23. Watermark δεν είναι data retention policy

Μην το μπερδέψεις με:

</> Plain text

```
VACUUM
retention
table TTL
```

Watermark αφορά:

streaming state + late-data handling.

Δεν διαγράφει historical Delta table rows επειδή παλιώσανε.

## 24. Output modes

Για stateful aggregation, αποτέλεσμα μπορεί να αλλάζει ανά batch.

Έχεις τρεις βασικές modes:

Mode	Τι emitάει
Append	μόνο rows που θεωρούνται πλέον final
Update	rows των οποίων το αποτέλεσμα άλλαξε στο trigger
Complete	ολόκληρο current state/result κάθε trigger

## 25. Append mode

Suppose:

</> Plain text

```
10:00-10:10 revenue
```

Στις 10:11 μπορεί να αλλάξει λόγω late data.

Άρα σε append mode δεν πρέπει να emitάρεις το final row αν μπορεί να αλλάξει αργότερα.

Με watermark:

```
</> Plain text
```

```
wait until window considered final
↓
emit once
```

Για window aggregations, append mode γράφει αφού περάσει το watermark threshold. [Databricks Docu...](#) +1

## 26. Update mode

Example state:

Batch 1:

```
</> Plain text
```

```
GR revenue = 100
```

emit:

```
</> Plain text
```

```
GR 100
```

Batch 2 receives +50:

```
</> Plain text
```

```
GR revenue = 150
```

emit:

```
</> Plain text
```

```
GR 150
```

Άρα downstream βλέπει updates.

Important current Databricks nuance:

Delta Lake sinks support append and complete modes, **not update mode directly**. Για Delta-like update semantics συνήθως

χρησιμοποιείς patterns όπως `foreachBatch + MERGE` . [Databricks Docu...](#)

Αυτό είναι καλό certification/interview detail.

## 27. Complete mode

Κάθε trigger:

</> Plain text

```
emit entire aggregate result
```

Example:

Batch 1:

</> Plain text

```
GR 100
```

```
DE 50
```

Batch 2:

</> Plain text

```
GR 150
```

```
DE 50
```

```
FR 20
```

γράφεται όλο το result.

Η Databricks επισημαίνει ότι σε complete mode το aggregation state δεν evictάρεται μέσω watermark όπως στο append/update pattern. [Databricks Docu...](#)

Άρα σε μεγάλα/unbounded cardinality aggregates μπορεί να είναι ακριβό.

## 28. Stateless streams και output modes

Για stateless operations:

</> Python

```
events.filter(...).select(...)
```

δεν έχεις stateful row που αλλάζει αργότερα.

Άρα append/update/complete distinction δεν έχει το ίδιο semantic effect· η current documentation λέει ότι για stateless operators οι output-mode distinctions ουσιαστικά δεν αλλάζουν operator behavior. [Databricks Docu...](#)

## 29. Checkpoint — τώρα πολύ βαθύτερα

Στη Phase 4 είπαμε:

</> Plain text

```
checkpoint
=
stream progress/state
```

Τώρα ανοίγουμε το κουτί.

Η current Databricks documentation λέει ότι checkpoint directory περιέχει τουλάχιστον:

</> Plain text

```
offsets
commits
state metadata/content
```

για stateful workloads. [Databricks Docu...](#)

Conceptually:

</> Plain text

```
checkpoint/
|
├─ offsets/
├─ commits/
├─ state/
└─ metadata
```

## 30. Offset

Source πρέπει να μπορεί να πει:

μέχρι πού έχω διαβάσει;

Kafka example:

</> Plain text

```
partition 0 → offset 1050
partition 1 → offset 2040
```

File/Auto Loader source έχει αντίστοιχο progress concept.

Checkpoint κρατά source offsets/process progress για κάθε micro-batch. [Databricks Docu...](#)

## 31. Commit log

Suppose micro-batch 42 processed source range X.

Checkpoint πρέπει επίσης να ξέρει:

αυτό το micro-batch γράφτηκε επιτυχώς στο sink;

Άρα υπάρχει commit information.

Databricks περιγράφει commits ως record των micro-batches που έχουν committed στο sink, βασικό κομμάτι exactly-once processing semantics. [Databricks Docu...](#)

## 32. State in checkpoint

Για stateful query:

</> Python

```
events.groupBy("customer_id").count()
```

πρέπει μετά restart να θυμάται:

</> Plain text

```
customer_1 count = 120
customer_2 count = 50
...
```

Αν restartάει χωρίς αυτό:

</> Plain text

```
new event for customer_1
```

δεν θα ξέρει ότι previous total ήταν 120.

Checkpoint therefore includes state-store metadata/content for stateful operators. [Databricks Docu...](#) +1

## 33. State store

Streaming state δεν είναι απλώς Python dictionary στον driver.

Είναι distributed state associated with streaming operators.

Η Databricks συνιστά σήμερα **RocksDB state store with changelog checkpointing** για demanding stateful workloads, ώστε state/checkpoint handling να scaleάρει καλύτερα και να μειώνεται checkpoint latency. [Databricks Docu...](#)

Δεν χρειάζεται ακόμη να ξέρεις RocksDB internals.

Mental model:

⟨/⟩ Plain text

```
stateful operator
 ↓
distributed state store
 ↓
checkpoint durability
```

## 34. Γιατί stateful query είναι πιο δύσκολο να αλλάξεις;

Έχεις checkpoint με state schema:

⟨/⟩ Plain text

```
key = customer_id
value = count
```

και αλλάζεις code σε:

⟨/⟩ Plain text

```
key = country
value = sum(amount)
```

Δεν μπορείς απαραίτητα να reuseάρεις το ίδιο checkpoint.

Η Databricks έχει explicit restrictions στο τι μπορεί να αλλάξει μεταξύ restarts από ίδιο checkpoint, ακριβώς επειδή checkpoint κωδικοποιεί query identity/state. [Databricks Docu...](#)

Άρα checkpoint δεν είναι generic reusable folder.

## 35. Deleting checkpoint

Αν διαγράψεις checkpoint:

⟨/⟩ Plain text

```
query identity/progress/state
→ gone
```

Η Databricks λέει ξεκάθαρα ότι deleting checkpoint ή changing checkpoint location κάνει το επόμενο run να ξεκινά σαν νέο query. [Databricks Docu...](#)

Αυτό μπορεί να σημαίνει:

⟨/⟩ Plain text

```
reprocessing
duplicate external side effects
loss of accumulated state
full rebuild required
```

ανά pipeline.

## 36. Exactly-once — τι σημαίνει πραγματικά;

Η φράση χρησιμοποιείται συχνά πολύ χαλαρά.

Για ένα managed Delta streaming sink:

</> Plain text

```
source offsets
+
checkpoint commits
+
Delta transaction log
```

συνεργάζονται ώστε retried micro-batch να μην γίνει partial duplicate commit.

Η Databricks αναφέρει ότι Delta streaming sink παρέχει exactly-once processing guarantees, ακόμα και με concurrent streams/batch queries. [Databricks Docu...](#)

## 37. Failure scenario

Suppose micro-batch 20 reads:

</> Plain text

```
events 1000–1100
```

και γράφει Delta.

System crashes ακριβώς στο boundary.

On restart πρέπει να αποφασίσει:

</> Plain text

```
Was batch 20 committed?
```

Checkpoint + Delta commit semantics επιτρέπουν:

</> Plain text

```
if committed:
 don't apply again
```

```
if not committed:
 retry
```

όχι:

```
</> Plain text

half applied twice
```

## 38. Exactly-once σταματά στα boundaries

Αυτό είναι πολύ σημαντικό.

Έστω:

```
</> Python

foreachBatch(send_email)
```

Micro-batch sends 100 emails.

Then process crashes before checkpoint commit.

On retry:

```
</> Plain text

same batch
→ send emails again
```

To external email service δεν έχει Delta transaction semantics.

Άρα:

```
</> Plain text

Structured Streaming exactly-once
≠
every arbitrary side effect exactly once
```

Η Databricks τονίζει ότι exactly-once guarantees στα managed Delta flows σταματούν στα edges, όπως custom/non-transactional sinks, και εκεί πρέπει να κάνεις το write idempotent. [Databricks Docu...](#) +1

## 39. Exactly-once ≠ no duplicate source events

Kafka/source μπορεί να στείλει:

</> Plain text

```
event_id=123
event_id=123
```

ως δύο distinct records.

Structured Streaming μπορεί να επεξεργαστεί κάθε input record exactly once και ακόμα:

</> Plain text

```
two duplicate business events
```

να φτάσουν στο table.

Άρα:

</> Plain text

```
processing guarantee
≠
business deduplication
```

Ακριβώς όπως μάθαμε με Auto Loader files.

## 40. Streaming deduplication

Simple:

</> Python

```
unique = events.dropDuplicates(["event_id"])
```

Αλλά σε unbounded stream υπάρχει πρόβλημα.

Για να ξέρεις αν `event_id=123` έχει εμφανιστεί ποτέ:

</> Plain text

```
πρέπει να θυμάσαι όλα τα event IDs για πάντα.
```

Unbounded state.

Γι' αυτό συνήθως χρειάζεσαι watermark/time bound.

Conceptually:

</> Python

```
unique = (
```

```
events
.withWatermark("event_time", "1 hour")
.dropDuplicates(["event_id", "event_time"])
)
```

Τώρα state μπορεί να καθαρίζεται με time progress.

Deduplication είναι stateful operation. [Databricks Docu...](#)

## 41. Stream-static join

Έχεις live orders:

</> Plain text

```
orders stream
```

και relatively static table:

</> Plain text

```
products
```

Μπορείς:

</> Python

```
products = spark.table("ecommerce.silver.products")

enriched = (
 orders_stream
 .join(products, "product_id")
)
```

Conceptually:

</> Plain text

```
stream events
+
static dimension
↓
enriched stream
```

Αυτό είναι πολύ common.

## 42. Caveat: static side semantics

Αν static table αλλάξει, συμπεριφορά εξαρτάται από query/execution semantics και refresh architecture.

Δεν πρέπει να υποθέτεις ότι ένα long-running stream magically recomputes all historical outputs επειδή dimension άλλαξε.

Αν correctness απαιτεί recomputation όταν dimension changes, materialized/declarative pipeline patterns μπορεί να είναι πιο κατάλληλα — αυτό θα φανεί στη Phase 6.

---

## 43. Stream-stream join

Έχεις:

```
</> Plain text
```

```
orders stream
payments stream
```

Θέλεις:

```
</> Plain text
```

```
order_id
```

match.

Problem:

Order event:

```
</> Plain text
```

```
10:00
```

Payment:

```
</> Plain text
```

```
10:02
```

Spark πρέπει να κρατήσει order state μέχρι να έρθει payment.

Αλλά πόσο;

Για πάντα;

Χωρίς watermark/time condition, state μπορεί να μεγαλώνει χωρίς bound.

---

## 44. Stream-stream join with watermarks

Conceptually:

**</> Python**

```
orders_w = orders.withWatermark(
 "order_time",
 "30 minutes"
)

payments_w = payments.withWatermark(
 "payment_time",
 "30 minutes"
)
```

και join condition περιλαμβάνει key + time relationship.

**</> Plain text**

```
same order_id
+
payment occurs within expected time range
```

Η Databricks συνιστά watermarks και στα δύο inputs για stream-stream joins ώστε να μπορεί να απορρίπτει obsolete state. Outer streaming joins απαιτούν watermarking. [Databricks Docu...](#)

## 45. Multiple watermarks

Αν δύο input streams έχουν:

**</> Plain text**

```
orders: 10 min delay
payments: 1 hour delay
```

Spark πρέπει να υπολογίσει global watermark.

Default Databricks/Spark policy είναι:

**</> Plain text**

```
minimum
```

δηλαδή ο πιο αργός stream καθορίζει progress.

Αυτό είναι safer για correctness.

Μπορείς να χρησιμοποιήσεις `max`, αλλά τότε slow-stream records μπορεί να dropped. Η Databricks προειδοποιεί να χρησιμοποιείται με προσοχή. [Databricks Docu...](#)

## 46. Why "min" makes sense

Suppose:

</> Plain text

```
orders advanced to 14:00
payments only 13:00
```

Αν χρησιμοποιήσεις orders watermark μόνο:

</> Plain text

```
Spark may conclude old state is safe to discard
```

ενώ payment events μπορεί ακόμα να εμφανιστούν.

Using minimum:

</> Plain text

```
global progress follows slowest input
```

protects correctness.

## 47. Source → transformation → sink

Ένα full stream έχει τρία μέρη.

</> Plain text

```
SOURCE
↓
TRANSFORMATION
↓
SINK
```

Example:

</> Python

```
events = (
 spark.readStream
 .table("ecommerce.bronze.events")
)

clean = (
 events
 .filter("event_type IS NOT NULL")
)

query = (
 clean.writeStream
 .option("checkpointLocation", checkpoint)
 .toTable("ecommerce.silver.events")
)
```

```
)
```

## 48. Delta as streaming source

Αν:

</> Plain text

```
Bronze Delta table
```

λαμβάνει continuously new rows, μπορείς:

</> Python

```
bronze = (
 spark.readStream
 .table("ecommerce.bronze.events")
)
```

και να streamάρεις additions downstream.

Delta tables μπορούν να χρησιμοποιηθούν και ως sources και sinks Structured Streaming. [Databricks Docu...](#)

## 49. Important Delta source caveat

Ένας simple Delta streaming source είναι naturally append-oriented.

Αν source Delta table έχει:

</> Plain text

```
UPDATE
DELETE
MERGE
```

τότε downstream semantics χρειάζονται περισσότερη προσοχή.

Για row-level changes, **Change Data Feed** είναι συχνά το σωστό tool.

Θα το δούμε βαθύτερα όταν ενώνουμε streaming με CDC.

## 50. Source retention caveat

Αν streaming query διαβάξει Delta source αλλά μένει stopped για πολύ περισσότερο από source retention:

</> Plain text

```
source files/history vacuumed
```

μπορεί να μην μπορεί να catch up.

Η current Databricks documentation λέει ότι stream πρέπει να τρέξει τουλάχιστον μία φορά μέσα στο source retention window· αλλιώς μπορεί να αποτύχει με missing-file error και να χρειάζεται reset/full refresh. [Databricks Docu...](#)

Αυτό είναι πραγματικό production operational requirement.

## 51. Don't "fix" missing stream files with ignoreMissingFiles

Η Databricks λέει συγκεκριμένα:

μην χρησιμοποιήσεις `spark.sql.files.ignoreMissingFiles=true` ως workaround σε stream που έχει μείνει πίσω από Delta retention, γιατί μπορεί να παραγάγει silent incorrect results. [Databricks Docu...](#)

Correct fix:

`</>` Plain text

```
increase retention
or
recover/rebuild stream appropriately
```

όχι suppress correctness error.

## 52. foreachBatch

Πολύ σημαντικό bridge μεταξύ streaming και batch APIs.

`</>` Python

```
def process_batch(batch_df, batch_id):
 ...

query = (
 stream_df.writeStream
 .foreachBatch(process_batch)
 .option("checkpointLocation", checkpoint)
 .start()
)
```

Για κάθε micro-batch:

`</>` Plain text

```
stream
↓
batch DataFrame
↓
your function
```

Άρα μέσα στη function μπορείς να χρησιμοποιήσεις batch-only operations όπως:

**</> Plain text**

```
MERGE
complex Delta DML
custom sink logic
```

## 53. Streaming MERGE pattern

**</> Python**

```
from delta.tables import DeltaTable

def upsert(batch_df, batch_id):
 target = DeltaTable.forName(
 spark,
 "ecommerce.silver.customers"
)

 (
 target.alias("t")
 .merge(
 batch_df.alias("s"),
 "t.customer_id = s.customer_id"
)
 .whenMatchedUpdateAll()
 .whenNotMatchedInsertAll()
 .execute()
)
```

Then:

**</> Python**

```
query = (
 changes.writeStream
 .foreachBatch(upsert)
 .option("checkpointLocation", checkpoint)
 .start()
)
```

This is a classic streaming-upsert pattern.

## 54. But foreachBatch must be idempotent

Suppose batch 10 retries.

Your function may execute again.

If function does:

```
</> Python
```

```
balance = balance + incoming_amount
```

twice, wrong.

If it does:

```
</> Plain text
```

```
MERGE by stable business key/version
```

it can be idempotent.

Therefore:

`foreachBatch` gives power, but you become responsible for sink-side idempotency.

---

## 55. batch\_id can help

`foreachBatch` receives:

```
</> Python
```

```
batch_id
```

which can be used in some external sink patterns to recognize replayed batches.

But don't treat `batch_id` alone as universal business dedup key.

The safest design depends on target system.

---

## 56. Real-time mode

Classic Databricks now also has a real-time Structured Streaming mode for ultra-low-latency operational workloads, targeting sub-second end-to-end latency in supported scenarios. [Databricks Docu...](#) +1

But this is **not the default thing you should choose for ordinary Data Engineering**.

For normal lakehouse ETL:

```
</> Plain text
```

```
micro-batch
or
AvailableNow
```

is usually simpler, cheaper and operationally easier.

## 57. Latency vs throughput

Smaller micro-batches:

</> Plain text

every 1 sec

Advantages:

</> Plain text

lower latency

Costs:

</> Plain text

more scheduling overhead  
more frequent commits  
more storage API calls  
smaller output files

Larger:

</> Plain text

every 1 min

Advantages:

</> Plain text

better throughput efficiency  
larger batches  
less overhead

Costs:

</> Plain text

higher latency

Streaming design is always balancing:

</> Plain text

latency  
throughput  
cost

```
state size
correctness
```

## 58. Small files in streaming

If every tiny micro-batch writes Delta:

⟨/⟩ Plain text

```
batch 1 → 2 MB file
batch 2 → 3 MB file
batch 3 → 1 MB file
...
```

You can create small-file pressure.

Delta can later compact files, and predictive optimization/OPTIMIZE can help, but latency requirements should still be chosen sensibly. Delta streaming integration is designed to address small-file and transactional concerns, but physical maintenance still matters. [🌐 Databricks Docu...](#)

## 59. Streaming backpressure / backlog

Suppose source produces:

⟨/⟩ Plain text

```
50 MB/sec
```

pipeline can process:

⟨/⟩ Plain text

```
20 MB/sec
```

Then:

⟨/⟩ Plain text

```
+30 MB/sec backlog
```

No checkpoint magic can fix throughput mismatch.

Eventually:

⟨/⟩ Plain text

```
latency ↑
backlog ↑
possibly source retention exceeded
```

Need:

⟨⟩ Plain text

```
more processing capacity
better query
larger batches
less state
better partitioning
reduce skew
```

## 60. Monitoring streaming query

At code level:

⟨⟩ Python

```
query.status
```

and:

⟨⟩ Python

```
query.lastProgress
```

can expose progress information.

In Spark UI / Databricks monitoring you care about things like:

⟨⟩ Plain text

```
input rows
processed rows/sec
input rows/sec
batch duration
state rows
watermark
shuffle
task skew
```

The current Databricks streaming concepts explicitly position Spark UI and `StreamingQueryListener` as production monitoring mechanisms. [🌐 Databricks Docu...](#)

## 61. Important metric: input rate vs processing rate

Suppose:

</> Plain text

```
inputRowsPerSecond = 50,000
processedRowsPerSecond = 100,000
```

pipeline has capacity.

If:

</> Plain text

```
input = 100,000
processed = 40,000
```

sustained:

</> Plain text

```
backlog grows
```

This is often more useful than simply looking at cluster CPU.

---

## 62. State growth debugging

Suppose every hour batch duration increases:

</> Plain text

```
10 sec
12 sec
20 sec
40 sec
2 min
...
```

while input rate stable.

Inspect stateful operators.

Maybe:

</> Plain text

```
numRowsTotal in state
```

keeps increasing.

Likely:

⟨/⟩ Plain text

```
missing watermark
watermark too long
unbounded key cardinality
bad stateful join
```

## 63. Watermark debugging

Suppose late events are missing.

Ask:

⟨/⟩ Plain text

```
What event_time did they carry?
What was max event time already observed?
What watermark delay was configured?
Had watermark already passed?
```

Do not reason only from:

⟨/⟩ Plain text

```
arrival time
```

Watermark is event-time-based.

## 64. "Late" is relative to query progress

Event arrives 20 minutes after it occurred.

Is it late?

Maybe.

If stream's maximum seen event time is only 5 minutes ahead:

⟨/⟩ Plain text

```
could still be accepted
```

If stream has already observed events 2 hours ahead:

⟨/⟩ Plain text

```
could be too late
```

Therefore:

</> Plain text

```
late
#
now - event_time only
```

It depends on event-time watermark progress.

## 65. Production example — event analytics

Source schema:

</> Plain text

```
event_id STRING
user_id BIGINT
event_type STRING
event_time TIMESTAMP
```

Read:

</> Python

```
events = (
 spark.readStream
 .table("ecommerce.bronze.events")
)
```

Aggregate:

</> Python

```
from pyspark.sql import functions as F

counts = (
 events
 .withWatermark(
 "event_time",
 "20 minutes"
)
 .groupBy(
 F.window(
 "event_time",
 "5 minutes"
),
 "event_type"
)
 .count()
)
```

Write:

</> Python

```
query = (
 counts.writeStream
 .outputMode("append")
 .option(
 "checkpointLocation",
 "/Volumes/ecommerce/system/event_counts_ckpt"
)
 .toTable("ecommerce.gold.event_counts")
)
```

## 66. What happens here?

Suppose events:

</> Plain text

```
10:01 click
10:02 click
10:03 purchase
```

Window:

</> Plain text

```
10:00-10:05
```

Spark keeps state:

</> Plain text

```
click = 2
purchase = 1
```

If event:

</> Plain text

```
event_time 10:04
```

arrives later, window state updates.

Eventually watermark advances sufficiently far.

Then append mode says:

</> Plain text

```
window final
→ emit result
```

→ state can be removed

That is the complete lifecycle.

## 67. Why append output can look "delayed"

User:

Events arrive instantly, why doesn't result appear instantly?

Because:

</> Plain text

```
stateful aggregate
+
append mode
+
watermark
```

means Spark must wait until it considers the aggregate final.

This is intended behavior, not necessarily lag.

## 68. Update-like result to Delta

If you want dashboard to see aggregate change continuously:

</> Plain text

```
10:01 → click_count=1
10:02 → click_count=2
```

Delta doesn't directly support streaming `update` output mode as a sink. Current Databricks guidance is to implement update-like semantics via `foreachBatch` + `MERGE` where needed. [Databricks Docu...](#)

Example:

</> Python

```
def merge_counts(batch_df, batch_id):
 target = DeltaTable.forName(
 spark,
 "ecommerce.gold.live_event_counts"
)

 (
 target.alias("t")
 .merge(
 batch_df.alias("s"),
 """
```

```
 t.window_start = s.window.start
 AND t.event_type = s.event_type
 """
)
 .whenMatchedUpdateAll()
 .whenNotMatchedInsertAll()
 .execute()
)
```

## 69. Streaming source and retention

Imagine:

⌕ Plain text

```
Friday:
stream stops

source Delta VACUUM retention:
7 days

two weeks later:
restart
```

Some source data needed to bridge Friday → now may be gone.

Checkpoint knows:

⌕ Plain text

```
I need version X
```

but physical file no longer exists.

Result:

⌕ Plain text

```
stream cannot recover incrementally
```

This is why streaming operations and source retention policy must be designed together. [Databricks Docu...](#)

## 70. Checkpoint and code deployment

Production deployment changes code.

Safe changes:

⌕ Plain text

```
some stateless filters/projections
```

may often reuse checkpoint depending on exact query.

Dangerous changes:

⌕ Plain text

```
state keys
state schema
source type
number/type of stateful operators
```

can be checkpoint-incompatible.

Rule:

Never assume "same checkpoint" means "new code will pick up seamlessly".

Consult compatibility rules and test stateful deployments carefully. [Databricks Docu...](#)

## 71. Stateful restart from new checkpoint

If you change state architecture:

⌕ Plain text

```
old checkpoint incompatible
```

you may need:

⌕ Plain text

```
new checkpoint
+
full recomputation/backfill
```

This is expensive, which is why state schema is effectively production data.

## 72. Micro-batch + Spark performance

A streaming micro-batch can suffer from the exact same issues as batch:

⌕ Plain text

```
skew
shuffle
bad join
too few partitions
too many partitions
```

```
spill
executor OOM
driver OOM
```

The difference is that streaming adds another dimension:

```
</> Plain text

must finish fast enough to keep up with arrivals
```

## 73. Example skew in stream

99 partitions:

```
</> Plain text

finish in 2 sec
```

1 partition:

```
</> Plain text

takes 30 sec
```

Trigger intended:

```
</> Plain text

5 sec
```

Now every micro-batch may take:

```
</> Plain text

30+ sec
```

Backlog grows.

This is the connection between Phase 1 Compute and Phase 5 Streaming.

## 74. Stateful streaming + skew is especially bad

Suppose one key:

```
</> Plain text

user_id = "anonymous"
```

contains 70% of events.

GroupBy:

</> Python

```
.groupBy("user_id")
```

creates hot/skewed state partition.

Possible symptoms:

</> Plain text

```
one task slow
one state partition huge
spill
increasing latency
```

So streaming partition/key design matters.

## 75. Current state-store best practice

For larger stateful workloads, Databricks currently recommends RocksDB with changelog checkpointing because it can reduce state checkpoint overhead and latency. [Databricks Docu...](#)

You don't need to configure it blindly.

You need to recognize:

When state becomes the bottleneck, compute sizing alone isn't necessarily the fix; the state-store architecture may matter.

## 76. Serverless streaming nuance

This is a useful 2026 detail.

Serverless notebooks/jobs:

</> Plain text

```
AvailableNow ✓
Once ✓ deprecated
ProcessingTime ✗
RealTime ✗
```

For always-running serverless streams:

</> Plain text

Lakeflow pipeline continuous mode

is the preferred platform architecture. [Databricks Docu...](#) +1

That directly connects this phase to **Phase 6 Lakeflow Pipelines**.

## 77. Practical lab 1 — rate source

For learning, Spark has a synthetic source:

</> Python

```
events = (
 spark.readStream
 .format("rate")
 .option("rowsPerSecond", 10)
 .load()
)
```

It produces approximately:

</> Plain text

timestamp	value
...	0
...	1
...	2

Use:

</> Python

```
transformed = (
 events
 .withColumn(
 "group_id",
 F.col("value") % 5
)
)
```

Then display/write it.

This removes external ingestion complexity while learning streaming.

## 78. Practical lab 2 — stateless

</> Python

```
positive = (

```

```
events
.filter("value % 2 = 0")
.select("timestamp", "value")
)
```

Observe:

</> Plain text

no aggregate state

Every record can be handled independently.

## 79. Practical lab 3 — stateful aggregate

</> Python

```
counts = (
 events
 .withColumn(
 "group_id",
 F.col("value") % 5
)
 .groupBy("group_id")
 .count()
)
```

Now Spark must remember:

</> Plain text

0 → count  
1 → count  
...

Inspect query progress/state metrics.

## 80. Practical lab 4 — event-time windows

</> Python

```
windowed = (
 events
 .withWatermark(
 "timestamp",
 "30 seconds"
)
 .groupBy(
 F.window("timestamp", "10 seconds")
)
)
```

```
.count()
)
```

Observe when windows appear/finalize under different output modes.

## 81. Practical lab 5 — AvailableNow on Delta

Suppose Bronze receives rows.

```
</> Python

stream_df = (
 spark.readStream
 .table("ecommerce.bronze.orders")
)
```

Then:

```
</> Python

(
 stream_df.writeStream
 .trigger(availableNow=True)
 .option(
 "checkpointLocation",
 "/Volumes/ecommerce/system/orders_to_silver"
)
 .toTable("ecommerce.silver.orders_incremental")
)
```

Run again with no new data.

Expected:

```
</> Plain text

nothing new to process
```

because checkpoint remembers progress. [Databricks Docu...](#)

## 82. Practical lab 6 — intentional checkpoint reset

Only in sandbox.

Run stream.

Then use a **new checkpoint location**.

Observe:

</> Plain text

```
query behaves as a new stream
```

This demonstrates why checkpoint is part of query identity. [Databricks Docu...](#)

Do not do this casually in production.

## 83. Practical lab 7 — late data

Create controlled events:

</> Plain text

```
arrival order:
10:01 event_time=10:01
10:02 event_time=10:02
10:20 event_time=10:03
```

Try:

</> Plain text

```
watermark = 5 min
```

and:

</> Plain text

```
watermark = 30 min
```

Observe difference in whether late data can still affect state.

That experiment will teach watermarking better than memorization.

## 84. Practical lab 8 — foreachBatch

</> Python

```
def inspect_batch(df, batch_id):
 print(f"Batch: {batch_id}")
 print(f"Rows: {df.count()}")

query = (
 events.writeStream
 .foreachBatch(inspect_batch)
 .start()
)
```

Now you will physically see:

</> Plain text

```
Batch 0
Batch 1
Batch 2
...
```

This makes micro-batches concrete.

## 85. Production debugging tree

When a stream is slow:

</> Plain text

STREAMING LATENCY HIGH

↓

Is input backlog growing?

↓

yes

↓

Which micro-batch is slow?

↓

Spark stage/task metrics

↓

shuffle	state	source/sink
skew	growth	throughput
spill	watermark	API latency
join	state store	storage

Streaming adds state/source/sink factors on top of ordinary Spark performance.

## 86. Debugging: stream stopped making progress

Check:

</> Plain text

```
query active?
source still producing?
new offsets/files available?
checkpoint corrupted?
sink blocked?
state batch stuck?
executor failure?
permission expired?
source retention violated?
```

Don't restart blindly before inspecting progress.

---

## 87. Debugging: increasing batch durations

If:

</> Plain text

input volume stable

but:

</> Plain text

batch duration grows

strong possibilities:

</> Plain text

state growth  
checkpoint overhead  
small files/output overhead  
source metadata growth  
sink slowdown

If state metrics grow endlessly:

</> Plain text

watermark/state design

is primary suspect.

---

## 88. Debugging: missing late records

Check:

</> Plain text

event\_time  
max observed event\_time  
configured watermark  
watermark progress  
output mode  
window boundaries

Do not first blame:

Plain text

```
checkpoint
```

unless evidence points there.

## 89. Debugging: duplicates after failure

Ask:

Plain text

```
Delta sink?
custom sink?
foreachBatch?
source itself duplicates?
checkpoint reset?
non-idempotent side effect?
```

Exactly-once semantics are end-to-end only where all pieces participate correctly.

## 90. Debugging: state store huge

Check:

Plain text

```
number of keys
watermark
window duration
late-data tolerance
join time bounds
dedup horizon
stateful key skew
```

Possible fix isn't necessarily bigger compute.

Maybe state is semantically unbounded.

## 91. Best-practice mental model

For production streaming, design these together:

Plain text

```
SOURCE
What are ordering/duplicate guarantees?

EVENT TIME
```

Which timestamp represents reality?

LATE DATA

How late can valid records arrive?

STATE

What information must be remembered?

WATERMARK

When can old state safely disappear?

TRIGGER

How much latency do we need?

SINK

Can it commit idempotently?

CHECKPOINT

Where is durable progress/state stored?

RETENTION

Those are not independent choices.

## 92. Certification trap: read vs readStream

Plain text

`spark.read`

→ bounded batch read

`spark.readStream`

→ streaming source

Simple but frequently tested.

## 93. Certification trap: processing time vs event time

Processing time:

when Databricks processes it.

Event time:

timestamp associated with when event occurred.

Watermarks generally use **event time**, not arrival wall clock. [Databricks Docu...](#)

## 94. Certification trap: watermark purpose

Correct:

</> Plain text

```
bound state
+
define tolerated lateness/finalization
```

Not:

</> Plain text

```
delete old table rows
```

## 95. Certification trap: checkpoint

Checkpoint stores:

</> Plain text

```
progress offsets
commits
state information
```

not just:

</> Plain text

```
"last timestamp"
``` :chatgpt-content-reference{index="42"}

---

# 96. Certification trap: AvailableNow

```text
process currently available backlog incrementally
then stop
```

not:

</> Plain text

```
run forever
``` :chatgpt-content-reference{index="43"}

---

# 97. Certification trap: serverless trigger

In current serverless notebooks/jobs:
```

```
```text
AvailableNow
```

is the important supported/recommended incremental trigger.

Time-based `ProcessingTime` trigger isn't supported there; always-on serverless streaming is better handled with Lakeflow pipeline continuous mode. [🌐 Databricks Docu...](#)

## 98. Certification trap: output mode

Append:

```
</> Plain text

finalized rows
```

Update:

```
</> Plain text

changed rows
```

Complete:

```
</> Plain text

entire state/result
```

But not every sink supports every mode. Delta sink does **not** directly support update output mode. [🌐 Databricks Docu...](#)

## 99. Certification trap: watermarks and joins

For stream-stream outer joins:

```
</> Plain text

watermark required
```

because engine must know when unmatched records can be considered final. [🌐 Databricks Docu...](#)

## 100. Big Phase 5 test

### Q1

Why can Spark Structured Streaming reuse batch-like DataFrame syntax?

**Answer:** Because Structured Streaming uses Spark's DataFrame/query-planning execution model incrementally over

unbounded input.

## Q2

What is a micro-batch?

**Answer:** A bounded chunk of newly available stream data executed as a Spark job.

## Q3

Does `readStream` immediately start processing?

**Answer:** No. It defines a streaming source/plan; a streaming sink/start operation initiates execution.

## Q4

What is `AvailableNow` ?

**Answer:** Process all currently available unprocessed input incrementally and then stop. [Databricks Docu...](#)

## Q5

Why is it useful?

**Answer:** Scheduled incremental jobs get checkpointed streaming semantics without an always-on process.

## Q6

Stateless vs stateful?

**Answer:** Stateless processing does not need historical intermediate state; stateful processing does.

## Q7

Is `groupBy` stateful?

**Answer:** In streaming aggregation, yes.

## Q8

Why?

**Answer:** Aggregate result must accumulate information across micro-batches.

## Q9

What is event time?

**Answer:** When the event occurred according to event data.

## Q10

What is processing time?

**Answer:** When the processing engine observes/processes it.

## Q11

Why can event time be earlier?

**Answer:** Network delays, offline clients, queues, retries and backlogs.

**Q12**

What does watermark solve?

**Answer:** Bounds state and sets a threshold for late-data handling/finalization. [Databricks Docu...](#)

**Q13**

Does 10 minutes mean exactly wall-clock event age?

**Answer:** No. It is relative to observed maximum event-time progress. [Databricks Docu...](#)

**Q14**

Short watermark advantage?

**Answer:** Less state and lower latency.

**Q15**

Disadvantage?

**Answer:** Lower tolerance for late data.

**Q16**

What does checkpoint contain?

**Answer:** Offsets/progress, commit information, query metadata and state information where applicable. [Databricks Docu...](#)

**Q17**

What happens if checkpoint deleted?

**Answer:** Query starts as new and loses prior progress/state identity. [Databricks Docu...](#)

**Q18**

Exactly-once Delta sink means source cannot contain duplicates?

**Answer:** No.

**Q19**

Why not?

**Answer:** Processing guarantees don't deduplicate semantically duplicated input records.

**Q20**

Why can `foreachBatch` duplicate external side effects?

**Answer:** A failed/retried batch may invoke the function again; external sinks may not participate in Spark/Delta transaction semantics.

**Q21**

Solution?

**Answer:** Make `foreachBatch` side effects idempotent, often by business key/version or transactional sink semantics.

**Q22**

Append mode?

**Answer:** Emits rows once considered final.

### Q23

Update mode?

**Answer:** Emits changed aggregate rows each trigger.

### Q24

Complete?

**Answer:** Emits complete aggregate state each trigger.

### Q25

Does Delta streaming sink support update output mode directly?

**Answer:** No; current Databricks docs list append and complete for Delta, with update-like semantics implemented via other patterns such as streaming MERGE. [Databricks Docu...](#)

### Q26

Why do stream-stream joins need state?

**Answer:** Record from one stream may need to wait for a later matching record from the other.

### Q27

Why watermarks?

**Answer:** To decide when waiting state can be discarded.

### Q28

Why can a stream fail if stopped beyond Delta retention?

**Answer:** Required source versions/files may have been vacuumed before the stream catches up. [Databricks Docu...](#)

### Q29

Should you set `ignoreMissingFiles=true` ?

**Answer:** No. Databricks warns this can silently produce incorrect streaming results. [Databricks Docu...](#)

### Q30

Input rate permanently exceeds processing rate. What happens?

**Answer:** Backlog and latency grow.

### Q31

Would checkpoint fix this?

**Answer:** No. Checkpoint preserves correctness/progress; it does not create processing capacity.

### Q32

Batch durations increase while input rate stays stable. What should you inspect?

**Answer:** State growth, checkpoint/state-store overhead, sink/source performance and Spark execution.

### Q33

One state partition dominates. Likely issue?

**Answer:** Key skew.

### Q34

What state-store technology does Databricks currently recommend for demanding stateful workloads?

**Answer:** RocksDB with changelog checkpointing. [Databricks Docu...](#)

### Q35 — interview

**"How would you explain Structured Streaming?"**

A strong answer:

Structured Streaming treats an unbounded data source as an incrementally growing table. The developer defines transformations using Spark's DataFrame model, while the engine executes newly available data in micro-batches or supported low-latency modes. Checkpoints track source progress and commits, and state stores preserve intermediate state for aggregations, joins and deduplication. Watermarks bound that state and define lateness tolerance for event-time processing.

---

## Phase 5 mini-project

Extend our project:

⟨⟩ Plain text

```
Auto Loader
 ↓
ecommerce.bronze.events
 ↓
Structured Streaming
 ↓
clean / validate
 ↓
ecommerce.silver.events
 ↓
window aggregation
 ↓
ecommerce.gold.events_5min
```

Bronze → Silver:

⟨⟩ Python

```
bronze = (
 spark.readStream
 .table("ecommerce.bronze.events")
)
```

```

silver = (
 bronze
 .filter("event_id IS NOT NULL")
 .filter("event_time IS NOT NULL")
)

(
 silver.writeStream
 .trigger(availableNow=True)
 .option(
 "checkpointLocation",
 "/Volumes/ecommerce/system/bronze_to_silver_events"
)
 .toTable("ecommerce.silver.events")
)

```

Gold:

</> Python

```

gold = (
 spark.readStream
 .table("ecommerce.silver.events")
 .withWatermark(
 "event_time",
 "20 minutes"
)
 .groupBy(
 F.window("event_time", "5 minutes"),
 "event_type"
)
 .agg(
 F.count("*").alias("events")
)
)

```

Then:

</> Python

```

(
 gold.writeStream
 .outputMode("append")
 .option(
 "checkpointLocation",
 "/Volumes/ecommerce/system/events_gold_5min"
)
 .toTable("ecommerce.gold.events_5min")
)

```

Test it deliberately with on-time events, late events within watermark, very late events, stream restart, a new checkpoint in a sandbox, duplicate business events and a prolonged stop that approaches source retention.

That completes the concepts specified in **Phase 5** of your roadmap: `readStream` , `writeStream` , micro-batches, checkpoints, offsets, state, watermarks, event/processing time, late data, output modes, stateless/stateful transformations,